



Universidad Internacional de La Rioja
Escuela Superior de Ingeniería y Tecnología

Máster en Análisis y Visualización de Datos Masivos

**Diseño y Desarrollo de una Plataforma de
Monitorización Operativa Mediante
Cuadros de Mando Conectados a Sistemas
OCR y Automatización con Agentes**

Trabajo fin de estudio presentado por:	Efrel Armando López Cáceres José Miguel Torres Cámara Víctor Romero Pardeiro
Tipo de trabajo:	Desarrollo
Director:	Rafael Martínez Ranera
Fecha:	15 de julio de 2026

Resumen

El presente Trabajo de Fin de Máster propone el diseño e implementación de OpsInsight Analytics, una plataforma multiagente orientada a entornos operativos e industriales (aunque en este trabajo se validará exclusivamente en entornos controlados con datos sintéticos) cuyo objetivo principal es transformar información operativa dispersa —formularios online, incidencias en PDF, reportes manuscritos y manuales técnicos — en un sistema de analítica de datos estructurado, trazable y accionable. El problema de partida es la fragmentación del conocimiento de negocio en organizaciones de operación y mantenimiento, que impide construir una visión consolidada del comportamiento de los activos y dificulta la toma de decisiones basada en evidencias.

La contribución central de este trabajo no reside en la creación de un componente aislado, sino en la integración de una arquitectura multiagente local que materializa la cadena de valor del dato de extremo a extremo: desde la ingesta de fuentes no estructuradas hasta la generación de diagnósticos técnicos inteligentes mediante RAG, ofreciendo una solución reproducible y sin dependencias de la nube para el soporte a la decisión en planta. La solución se articula en cinco capas técnicas: ingesta multiformato, extracción mediante OCR y modelos de lenguaje local (Qwen2.5), validación determinista de calidad, diagnóstico mediante Generación Aumentada por Recuperación (RAG) y observabilidad industrial en tiempo real.

La orquestación de todo el pipeline se implementa mediante n8n, asegurando la soberanía del dato y la privacidad al ejecutarse íntegramente de forma local. La metodología sigue el marco CRISP-DM, adaptada al dominio industrial. Los resultados demuestran la viabilidad de pasar de documentación heterogénea a inteligencia operativa, logrando un repositorio consolidado en MySQL 8.0 y un *dashboard* en Grafana auto-provisionado que integra métricas de negocio y salud del sistema. La plataforma se validó de extremo a extremo sobre documentación sintética basada en los datasets de referencia AI4I 2020 y MetroPT-3.

Palabras clave: sistemas multiagente, analítica operacional, OCR, LLM, reconocimiento óptico de caracteres, modelos de lenguaje de gran escala.

Abstract

This Master's Thesis presents the design, implementation and validation of OpsInsight Analytics, a local multi-agent platform for operational and industrial environments (however, during this work, it will only be validated in a controlled environment with synthetic data) whose primary objective is to transform scattered operational information —online forms, PDF incident reports, handwritten reports and technical manuals— into a structured, traceable and actionable data analytics system. The starting problem is the fragmentation of business knowledge in operations and maintenance organisations, which prevents a consolidated view of asset behaviour and hinders evidence-based decision-making.

The central contribution of this work lies not in the creation of an isolated component, but in the integration of a local multi-agent architecture that implements the end-to-end data value chain: from the ingestion of unstructured sources to the generation of intelligent technical diagnostics using RAG, offering a reproducible and cloud-independent solution for decision support on the shop floor. The solution is structured across five technical layers: multi-format ingestion, extraction via OCR and local language models (Qwen2.5), deterministic quality validation, diagnostics via Retrieval-Augmented Generation (RAG), and real-time industrial observability.

The entire pipeline is orchestrated using n8n, ensuring data sovereignty and privacy as it runs entirely on-premises. The methodology follows the CRISP-DM framework, adapted to the industrial domain. The results demonstrate the feasibility of moving from heterogeneous documentation to operational intelligence, resulting in a consolidated repository in MySQL 8.0 and a self-provisioned Grafana dashboard that integrates business and system health metrics. The platform was validated end-to-end using synthetic documentation based on the AI4I 2020 and MetroPT-3 reference datasets.

Keywords: multi-agent systems, operational analytics, OCR, local LLM, RAG, data validation

Índice de contenido

1.	Introducción	1
1.1.	Motivación	1
1.2.	Planteamiento del trabajo	3
1.3.	Estructura del trabajo	4
2.	Contexto y estado del arte	6
2.1.	Contexto del problema	6
2.2.	Estado del arte	7
2.2.1.	LLMs locales y sistemas multiagentes	7
2.2.2.	<i>Optical Character Recognition (OCR)</i>	9
2.2.3.	<i>Dashboards</i>	12
2.3.	Análisis de datos de mantenimiento e incidencias	17
2.3.1.	Datasets de referencia en mantenimiento	17
2.3.2.	Limitaciones estructurales de los datos operativos reales	19
2.3.3.	Metodologías de análisis de fallos e incidencias en la literatura	19
2.4.	Conclusiones	20
3.	Objetivos concretos y metodología de trabajo.....	22
3.1.	Objetivo general.....	22
3.2.	Objetivos específicos	22
3.3.	Metodología del trabajo	24
3.3.1.	Marco metodológico: CRISP-DM	24
3.3.2.	Plan técnico de implementación	28
4.	Marco normativo.....	30
4.1.	Protección de datos personales.....	30
4.2.	Licencias de los conjuntos de datos.....	31

4.3.	Licencias de los modelos de inteligencia artificial	31
4.4.	Consideraciones éticas.....	31
5.	Desarrollo específico de la contribución	33
5.1.	Infraestructura y despliegue del sistema.....	34
5.2.	Arquitectura basada en agentes.....	36
5.2.1.	Agente de extracción documental	37
5.2.2.	Validación del OCR para ingesta documental	43
5.2.3.	Agente de análisis documental	53
5.2.4.	Módulo de validación de JSON	60
5.3.	Registro de incidencias	65
5.4.	Control de errores y trazabilidad del sistema.....	67
5.5.	Dashboard de monitorización.....	69
5.5.1.	Análisis e interpretación de los indicadores operativos	72
5.6.	Evaluación del sistema: validación y monitorización	73
5.6.1.	Evaluación unitaria del módulo de validación.....	73
5.6.2.	Verificación de extremo a extremo	76
5.7.	Discusión de los resultados.....	79
6.	Código fuente y datos analizados	81
6.1.	Código fuente.....	81
6.1.1.	Reproducibilidad y puesta en marcha	81
6.2.	Datos analizados	82
7.	Conclusiones.....	85
8.	Limitaciones y prospectiva	88
8.1.	Limitaciones	88
8.2.	Trabajo futuro	89

Referencias bibliográficas.....	91
9. Índice de acrónimos y abreviaturas	95

Índice de figuras

Figura 1 Evolución de la popularidad de los principales algoritmos de OCR (2021-2026)	11
Figura 2 Diagrama Gantt con las tareas planificadas	27
Figura 3 Esquema del funcionamiento del sistema y herramientas	34
Figura 4 Vista general del workflow OpsInsight en n8n	34
Figura 5 Módulo de ingesta documental en n8n	41
Figura 6 Módulo de análisis documental en n8n	53
Figura 7 Flujo operativo de los datos	54
Figura 8 Módulo de validación del JSON en n8n	63
Figura 9 Módulo de registro de incidencias en n8n	65
Figura 10 Workflow de manejo de errores OpsInsight_ErrorHandler en n8n	68
Figura 11 Dashboard de monitorización en Grafana con datos sintéticos de validación	70
Figura 12 Resultado de la suite de pruebas (19/19 superados)	76

Índice de tablas

Tabla 1 Estimación de costes utilizada por los agentes.	18
Tabla 2 Fases del proceso CRISP-DM.....	25
Tabla 3 Distribución de responsabilidades por miembro del equipo	28
Tabla 4 Servicios contenedorizados que componen OpsInsight Analytics	35
Tabla 5 Formatos de entrada	38
Tabla 6 Ejemplo de incidencia en formato JSON.....	39
Tabla 7 Prompt para generar un JSON	42
Tabla 8 Formulario impreso para validar la ingesta documental con OCR.	44
Tabla 9 JSONs obtenidos a partir de formularios totalmente impresos.	45
Tabla 10 Formulario híbrido para validar la ingesta documental con OCR.....	47
Tabla 11 JSONs obtenidos a partir de formularios parcialmente manuscritos.....	48
Tabla 12 Formulario manuscrito para validar la ingesta documental con OCR.	50
Tabla 13 JSONs obtenidos a partir de formularios totalmente manuscritos.....	51
Tabla 14 Configuración de los parámetros de inferencia	56
Tabla 15 Instrucciones técnicas críticas (System Message)	57
Tabla 16 Detalle de validación estructural	60
Tabla 17 Detalle de reglas para alertas	61
Tabla 18 Campos para registrar incidencias.....	64
Tabla 19 Tabla para registro de incidencias	68
Tabla 20 Paneles configurados en Grafana	70
Tabla 21 Casos de pruebas	74
Tabla 22 Casos capa semántica	75
Tabla 23 Escenarios de verificación de extremo a extremo.....	76

Mecanismos de coordinación empleados

El equipo emplea los siguientes mecanismos de coordinación y comunicación para el desarrollo del TFE:

- Reuniones mediante Google Meet, en las que se planifica y distribuye el trabajo restante, se revisa el avance de cada módulo, se toman decisiones técnicas conjuntas y se resuelven bloqueos.
- Repositorio común en GitHub, que centraliza el código fuente, los notebooks de análisis y los datos de ejemplo. Cada integrante trabaja en una rama propia y las incorporaciones al repositorio principal se realizan mediante *pull requests* revisados por al menos otro miembro del grupo.
- Documento de trabajo colaborativo en Microsoft Word 365, utilizado para la redacción conjunta de la memoria, la revisión cruzada entre secciones y el seguimiento del estado de cada apartado. Se mantiene un registro de versiones con fecha y autor de los cambios más significativos.
- Canal de mensajería instantánea (WhatsApp) para comunicaciones rápidas y coordinación operativa entre reuniones.
- Revisión cruzada de contenidos: cada integrante revisa y comenta el trabajo redactado por los demás antes de considerar un apartado como cerrado, garantizando la coherencia interna del documento.

1. Introducción

Las organizaciones de operación y mantenimiento industrial gestionan un volumen creciente de información operativa —partes de incidencia, órdenes de trabajo, manuales técnicos y registros de audio— que en su mayor parte permanece infrautilizada. Esta información, dispersa en múltiples formatos y sistemas, contiene un potencial analítico significativo: la capacidad de identificar activos críticos, anticipar patrones de fallo recurrente y fundamentar decisiones de mantenimiento en evidencias documentales trazables.

El presente trabajo propone OpsInsight Analytics, una plataforma multiagente local que transforma esa documentación heterogénea en un sistema de analítica operativa estructurado y accionable. La plataforma integra extracción documental mediante OCR, modelos de lenguaje locales, un repositorio analítico homogéneo, una base de datos de incidencias registradas y un cuadro de mando (*dashboard*) interactivo, todo ello implementado con herramientas de código abierto y sin dependencias de servicios en la nube.

Dada la dificultad de acceder a datos de monitorización y reportes de incidencias reales, este proyecto se ha desarrollado utilizando exclusivamente entornos controlados y manuales y reportes de incidencia sintéticos.

En los apartados siguientes se presenta la motivación del proyecto (1.1), el planteamiento y alcance de la solución propuesta (1.2) y la estructura del documento (1.3).

1.1. Motivación

El problema central que motiva este trabajo es, precisamente, la incapacidad de convertir esa información operativa dispersa en conocimiento analítico accionable, además de la capacidad de utilizar la tecnología actual para dar asistencia a los operarios para gestionar problemas rápido y ahorrar costes de soporte técnico y/o carga de trabajo a expertos.

Las organizaciones disponen de muchos datos, pero no de los mecanismos para integrarlos, estructurarlos y explotarlos conjuntamente. Como consecuencia, la gestión de activos e incidencias se apoya en la experiencia informal de los técnicos, en intuiciones construidas a lo largo del tiempo, y no en un análisis sistemático del comportamiento histórico de los equipos en conjunción con los detalles especificados en el manual y los modos de funcionamiento

“normal”. Se sabe, de manera informal, que una máquina "da muchos problemas" o que cierto tipo de fallo "aparece siempre en verano", pero esas percepciones rara vez pueden demostrarse con datos integrados, cuantificarse en términos de coste o tiempo de parada, ni traducirse en intervenciones preventivas justificadas.

Las causas de esta situación son múltiples y se retroalimentan entre sí. En primer lugar, la diversidad de formatos en los que se registran las incidencias, desde documentos escaneados hasta archivos tabulares sin esquema común, imposibilita la consolidación automática de la información sin una capa de extracción y normalización previa. En segundo lugar, los sistemas de gestión de mantenimiento existentes (CMMS, ERP, herramientas ITSM) fueron concebidos para el registro transaccional, no para el análisis exploratorio ni para la integración con documentación técnica no estructurada. En tercer lugar, la adopción de técnicas de inteligencia artificial y análisis de datos en el ámbito del mantenimiento industrial ha avanzado de forma desigual, mientras que la investigación académica ha producido resultados sólidos en predicción de fallos a partir de datos de sensores (Pech et al., 2021) (Zonta et al., 2020) la explotación analítica de documentación operativa heterogénea, (partes de incidencia, informes cualitativos, manuales técnicos) sigue siendo un área con escasa cobertura práctica y metodológica.

La relevancia del problema trasciende el ámbito operativo inmediato. Desde una perspectiva científica, existe un vacío metodológico en la integración de pipelines de extracción documental con analítica de datos masivos aplicada al mantenimiento industrial. La mayor parte de los trabajos existentes en mantenimiento y soporte técnico automatizados parten de datos estructurados, series temporales de sensores, registros tabulares de fallos, y no abordan el desafío de construir esa base analítica desde fuentes documentales heterogéneas (Taoufyq et al., 2025). Desde una perspectiva aplicada, la solución a este problema tiene un impacto directo y cuantificable: la reducción del tiempo medio de reparación y la priorización de activos críticos. La solución permite la reducción del tiempo medio de reparación (MTTR), la validación de coherencia semántica de los reportes y la mejora de la trazabilidad documental mediante el uso de inteligencia artificial local

Es en este contexto donde surge OpsInsight Analytics, una plataforma local orientada a transformar información operativa dispersa en un sistema analítico útil. La motivación no reside en la aplicación de inteligencia artificial como fin en sí mismo, sino en la construcción

de una cadena de valor del dato completa, desde la ingestión de documentos no estructurados hasta la visualización de patrones de fallo y la generación de evidencias para la toma de decisiones. Este trabajo se plantea, por tanto, como una contribución tanto al campo de la analítica de datos industriales como al de la extracción y estructuración de información técnica, con el objetivo de demostrar que es posible pasar de documentación operativa heterogénea a inteligencia analítica accionable mediante un enfoque reproducible, trazable y metodológicamente riguroso, tanto desde una perspectiva de negocio a través de análisis periódicos como de forma inmediata y operativa al generarse un fallo.

1.2. Planteamiento del trabajo

OpsInsight Analytics aborda el problema de la información operativa dispersa mediante la integración de cinco capacidades complementarias en un único pipeline de soporte a la decisión. El valor diferencial de la propuesta es su carácter de integración y de solución local, el sistema no solo visualiza datos, sino que orquesta la extracción (OCR/LLM), la validación determinista de calidad y el diagnóstico basado en conocimiento experto (RAG contra manuales técnicos). Esta integración permite transformar el conocimiento fragmentado y silencioso de los reportes manuales en inteligencia operativa accionable y trazable en tiempo real. En primer lugar, el sistema incorpora un módulo de ingesta y extracción documental capaz de procesar manuales técnicos y reportes de incidencias en múltiples formatos (PDF, imágenes, formularios escaneados, formularios web, etc.). Para procesar texto de imágenes, se aplicarán técnicas de reconocimiento óptico de caracteres (OCR) y modelos de lenguaje para transformar los documentos ingeridos en registros JSON estructurados y homogéneos. Estos dos primeros módulos constituyen la base del sistema para poder trabajar con datos limpios y estructurados.

El segundo módulo es el responsable de la normalización semántica que valida el contenido del JSON generado, unifica nomenclaturas, formatos de fechas, idiomas utilizados, etc. Esta estructura, también resuelve variantes terminológicas y relaciona cada incidencia con los fragmentos de manuales técnicos más relevantes mediante recuperación basada en similitud semántica. La conexión entre el historial de incidencias y el conocimiento documental representa una de las aportaciones diferenciales del proyecto. Para dar robustez al sistema se

desarrollan dos capas, una estructural (bloqueante) que asegura que los datos sean correctos, y una semántica que detecta inconsistencias contra el manual MetroPT-3.

En tercer lugar, la plataforma incluye una capa de análisis semántico que coteja las incidencias reportadas con los manuales técnicos relevantes, incluyendo estimación de costes y criticidad, los valores “normales” de las variables monitorizadas, procesos para resolver problemas recurrentes, etc. La plataforma integra un AI Agent que utiliza técnicas de RAG (Generación Aumentada por Recuperación) para cotejar las incidencias con el manual técnico en tiempo real, estimando criticidad, tiempos y costes de reparación sin 'alucinar' información fuera del manual. Esto se hace utilizando un LLM ejecutado en local, asegurando la privacidad de la información tratada.

La cuarta etapa es un registro de incidencias para su análisis posterior, tanto en un archivo CSV como en una base de datos. Esto servirá también como fuente de datos para el *dashboard*.

El último y quinto componente es un *dashboard* que presenta los resultados de forma interactiva para explorar indicadores clave, organizar los análisis por tipos de fallos, visualizar la evolución temporal de las incidencias y poder obtener sus evidencias documentales. El objetivo es demostrar que, a partir de documentación operativa heterogénea, es posible construir una herramienta de soporte a la decisión robusta, trazable y defendible como prototipo de producto mínimo viable, tanto para personal operativo de forma inmediata como para personal de gestión.

En el contexto de este TFM, los cinco módulos descritos (ingesta documental y OCR, validación/normalización de JSONs, recuperación y análisis de manuales, registro de incidencias y *dashboard*) se desarrollarán completamente hasta el punto en el que el sistema sea demostrable y pueda ser validado para detectar limitaciones y definir trabajos futuros.

1.3. Estructura del trabajo

Tras exponer la motivación y el planteamiento del proyecto, esta subsección se dispone a definir la estructura y el contenido de las siguientes secciones del trabajo.

La sección 2 contiene un estado del arte que enumerará las fuentes investigadas y tenidas como referencia para el desarrollo del trabajo, sirviendo a su vez para definir el contexto

tecnológico en el cual se desarrolla el mismo. Más en particular, se investiga el estado y uso actual de las tecnologías implicadas en el proyecto, incluyendo agentes de automatización basados en Inteligencia Artificial, algoritmos de análisis visual como *Optical Character Recognition* (OCR), el uso de bases de datos y la creación de *dashboards* para monitorización de tareas.

La sección 3 concretiza la definición de los objetivos del proyecto, tanto generales como específicos. También expone la metodología que se utiliza para alcanzarlos. Esto se completa con la sección 4, donde se detalla el marco normativo relativo a las regulaciones relevantes dado el contexto técnico del trabajo, así como los datos a utilizar y los objetivos buscados.

Las secciones siguientes son las más técnicas, ya que exponen el desarrollo e implementación del trabajo (sección 5) y los datos analizados (sección 6).

Finalmente, se exponen las conclusiones y las limitaciones en las secciones 7 y 8, respectivamente.

2. Contexto y estado del arte

2.1.Contexto del problema

La transformación digital de los entornos industriales ha generado una cantidad creciente de datos operativos que, en su mayor parte, permanecen infrautilizados. El paradigma de la Industria 4.0 ha impulsado la adopción masiva de sensores, sistemas SCADA (solución de hardware y software que permite supervisar, controlar y automatizar procesos industriales en tiempo real desde una ubicación centralizada) y plataformas de gestión de activos, pero la capacidad de convertir esos datos en conocimiento analítico accionable continúa siendo un reto no resuelto en la mayoría de las organizaciones (Pech et al., 2021). En el ámbito concreto del mantenimiento operativo, este reto se agudiza porque los datos no solo provienen de sensores estructurados, sino también de registros documentales heterogéneos, partes de avería en papel (impresas o escritas a mano), formularios web o PDFs exportados desde sistemas ERP, manuales técnicos de fabricante y notas de operario en texto libre.

La consecuencia directa es que el análisis de incidencias operativas, su frecuencia, criticidad, coste, evolución temporal y relación con procedimientos correctivos, rara vez se realiza de forma sistemática. La mayor parte de los trabajos de mantenimiento y soporte técnico basados en aprendizaje automático parten de conjuntos de datos ya estructurados, habitualmente series temporales de sensores, ignorando la dimensión documental del problema. Esta limitación es precisamente el punto de partida de OpsInsight Analytics: no trabajar sobre datos de sensor en tiempo real, sino sobre la riqueza analítica que contienen los registros operativos históricos una vez que se logra estructurarlos de forma homogénea.

Este problema es especialmente relevante en organizaciones de servicios tecnológicos como Sopra Steria, entorno de referencia de este proyecto. En estos contextos, la gestión de incidencias operativas se distribuye entre sistemas ITSM y documentación técnica en formatos no homogéneos. La ausencia de un pipeline integrado que consolide estas fuentes impide construir una visión analítica del comportamiento de los activos gestionados, limitando la capacidad de anticipar fallos recurrentes y priorizar intervenciones basadas en evidencias. OpsInsight Analytics se plantea como respuesta directa a este déficit.

2.2.Estado del arte

Esta sección investiga el estado y uso actual de las tecnologías implicadas en el proyecto, incluyendo:

- Agentes de automatización basados en Inteligencia Artificial para orquestar las tareas a realizar.
- Algoritmos de análisis visual como *Optical Character Recognition* (OCR) y LLMs para extraer la información de los documentos de entrada.
- Bases de datos para almacenar el registro histórico de incidencias.
- Creación de *dashboards* para monitorizar las tareas.

2.2.1. LLMs locales y sistemas multiagentes

El uso de modelos de lenguaje de gran tamaño (LLMs) para la extracción estructurada de información a partir de texto técnico no supervisado representa uno de los avances más relevantes en el procesamiento del lenguaje natural aplicado a dominios industriales. A diferencia de los enfoques clásicos basados en reglas o en Named Entity Recognition supervisado, los LLMs permiten obtener campos estructurados en formato JSON a partir de texto libre sin necesidad de un corpus anotado específico (Brown et al., 2020).

Touvron et al. (2023) demostraron con el lanzamiento de LLaMA que es posible obtener capacidades de extracción y razonamiento comparables a modelos mucho más grandes con menos de 13 mil millones de parámetros, abriendo la puerta al despliegue local en entornos con restricciones de privacidad. En la actualidad, modelos como Qwen2.5 de Alibaba o Mistral Small 3 ofrecen rendimiento suficiente para tareas de extracción de información técnica con 7B parámetros ejecutables en CPU con 5 GB de RAM mediante herramientas como Ollama (Ollama, 2023). En su versión de 3B de parámetros solo necesita 2 GB de RAM sin sacrificar demasiada precisión, lo que puede ser suficiente para tareas simples como las requeridas en este sistema (rellenar un JSON a partir de texto plano y sugerir una acción en función de un manual).

En cuanto a la orquestación de pipelines de extracción y análisis, herramientas como n8n (n8n GmbH, 2019) permiten construir flujos de trabajo (*workflows*) automatizados conectando módulos de OCR, LLM y análisis mediante una arquitectura de agentes con responsabilidades

diferenciadas. Este enfoque modular facilita la trazabilidad del pipeline y permite sustituir componentes individuales sin rediseñar el sistema completo. Frente a alternativas más cerradas, n8n ofrece flexibilidad de integración y compatibilidad con despliegue completamente local.

Entre las alternativas a n8n en el espacio de la orquestación de agentes con LLMs destaca Activepieces (Activepieces, 2022), una plataforma de automatización de código abierto que, al igual que n8n, permite construir *workflows* visuales conectando bloques funcionales. Activepieces es propuesta expresamente por Sopra Steria como herramienta de referencia en su entorno de automatización (Sopra Steria, 2025). Sin embargo, su ecosistema de integraciones nativas es aún más reducido que el de n8n —especialmente en conectores con herramientas de análisis de datos y modelos LLM locales— y su comunidad de desarrollo es menor, lo que limita la disponibilidad de plantillas y documentación técnica aplicable al dominio industrial. Por ello, se mantiene n8n como orquestador principal, sin descartar Activepieces como alternativa para nodos específicos de integración empresarial si el entorno de despliegue final de Sopra Steria así lo requiere.

Un aspecto crítico señalado por la literatura es la propagación de errores entre agentes: un dato mal extraído puede validarse erróneamente por el agente siguiente y presentarse como correcto en el *dashboard* sin que ningún agente individual detecte la inconsistencia (Dmitrievna & Parsadanyan, 2025). Para mitigar este riesgo se propone incorporar puntos de validación entre etapas clave del pipeline, especialmente en las transiciones extracción→normalización y normalización→análisis, bien mediante reglas deterministas, bien mediante un agente LLM de verificación cruzada.

En lo referente al repositorio de datos operativos, el análisis técnico de soluciones para la gestión de incidencias identifica dos paradigmas principales: los motores analíticos embebidos y los sistemas de gestión de bases de datos relacionales (RDBMS). Entre los primeros destaca DuckDB (Raasveldt & Mühleisen, 2019), una solución optimizada para cargas de trabajo OLAP (*Online Analytical Processing*) que utiliza almacenamiento orientado a columnas y ejecución vectorizada de consultas, permitiendo un alto rendimiento en agregaciones complejas sobre grandes volúmenes de datos sin necesidad de un servidor externo (DuckDB Foundation, 2024). Por otro lado, sistemas relacionales consolidados como MySQL 8.0 ofrecen una arquitectura cliente-servidor madura que garantiza la persistencia mediante el estricto cumplimiento de

las propiedades ACID. Mientras que DuckDB es ideal para el análisis ad-hoc de archivos masivos como CSV o Parquet, MySQL proporciona una gestión de concurrencia superior y una conectividad nativa universal con herramientas de orquestación de procesos y plataformas de monitorización operativa como Grafana. Para el diseño de *OpsInsight Analytics*, se ha seleccionado MySQL como motor de persistencia central debido a su robustez en la gestión de registros transaccionales, su capacidad para integrar metadatos derivados de la validación semántica y su estabilidad operativa en entornos industriales contenedorizados, donde la integridad del historial y la disponibilidad del dato para múltiples servicios concurrentes son requisitos prioritarios frente al rendimiento puramente analítico de los motores embebidos.

Además, el orquestador de agentes seleccionado (n8n), no dispone de conector nativo con DuckDB, todo lo contrario de MySQL que es soportado de forma nativa por el orquestador.

2.2.2. *Optical Character Recognition* (OCR)

2.2.2.1. Evolución del OCR

Durante la primera mitad del siglo XX, Emanuel Goldberg y Gustav Tauschek patentan dispositivos para distinguir caracteres (Goldberg, 1931; Tauschek, 1935) y, con GISMO, se transiciona del OCR mecánico al electrónico (History of Information, s. f.), dando paso a aplicaciones como MICR (1960), RCA Videoscan (1965) o la ordenación automática de correo (1966) (American Bankers Association, s. f.; US National Archives, 2015).
<https://patents.google.com/patent/US1838389A/en>
<https://patents.google.com/patent/US2026330A/en>
<https://www.historyofinformation.com/detail.php?entryid=885>
<https://www.aba.com/about-us/our-story/aba-history/1950-1974>
http://rca.vobj.org/RCA_Engineer/RCA_Engineer_v10/RCA_Engineer_v10n1/p44KlienMiller-Videoscan.pdf
<https://www.youtube.com/watch?v=V4LJs2ZoDR4>

El siglo termina con el desarrollo de Tesseract por HP (Smith, 2007) y con redes neuronales (LeNet) capaces de reconocer caracteres manuscritos con una precisión mayor al 99% (Lecun et al., 1998).
<https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/33418.pdf>
http://vision.stanford.edu/cs598_spring07/papers/Lecun98.pdf Esto hace que la industria considere el OCR resuelto y que lo aplique a gran escala, con ejemplos como Google

Books (1998) y ABBYY FineReader (2000) (ABBYY, s. f.; Google Books, s. f.).<https://books.google.com/https://pdf.abbyy.com/?srsltid=AfmBOooSZK3O2GqrRtMLIJPM7EjpVBcnd8bwpq2fEEMEfMn1AX0-uCPF> HP libera Tesseract en 2005 (con el patrocinio de Google).<https://github.com/tesseract-ocr/tesseract>

En 2012, [AlexNet](#) demuestra que las redes convolucionales superan a los algoritmos tradicionales (Krizhevsky et al., 2012). En 2017, se proponen [arquitecturas convolucionales y recurrentes](#) (RCNN) que se convierten en el nuevo estándar (Keren & Schuller, 2017). Más adelante se proponen EAST (Zhou et al., 2017) y CRAFT (Baek et al., 2019).<https://arxiv.org/abs/1704.03155https://arxiv.org/abs/1904.01941> En 2020, [PaddleOCR](#) se transformó en el nuevo estándar para las arquitecturas VLM (Vision Language Models) (Du et al., 2020) y en 2021 surgen modelos basados en transformers como TrOCR (Li et al., 2022) o Donut (Kim et al., 2022).<https://arxiv.org/abs/2109.10282https://arxiv.org/abs/2111.15664>

2.2.2.2. Implementaciones más relevantes

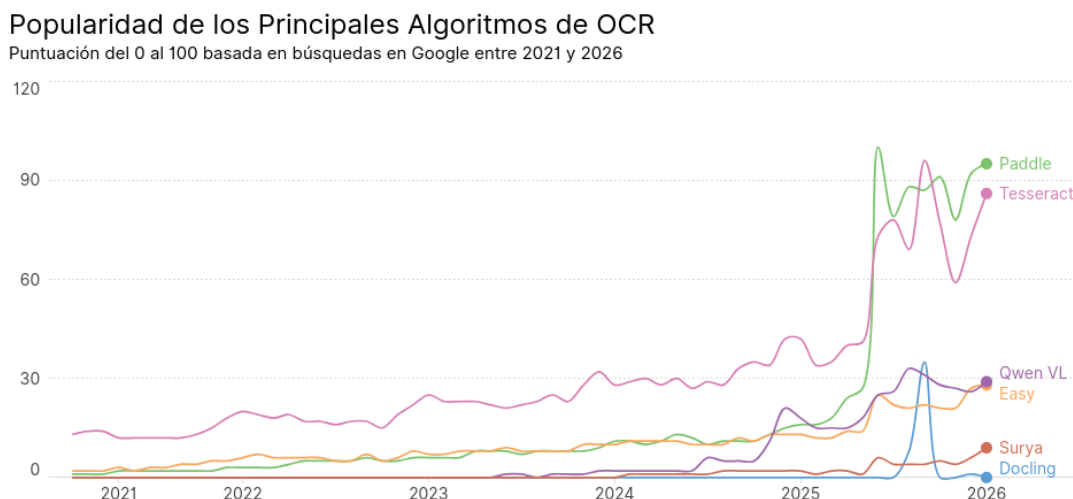
Para decidir qué algoritmo OCR utilizar, se cotejaron las recomendaciones de Sopra Steria con las opiniones de expertos de modal.com, unstract.com o javadex.es (Bispo, 2025; Criado, 2026; Modal, s. f.). Además, se analizó la evaluación hecha por codesota.com de varios algoritmos con ocho distintos benchmarks (Wikiel, s. f.).https://modal.com/blog/8-top-open-source-ocr-models-comparedhttps://unstract.com/blog/best-opensource-ocr-tools/https://www.javadex.es/blog/mejores-modelos-open-source-ocr-reconocimiento-texto-2026?utm_source=chatgpt.comhttps://www.codesota.com/ocr?utm_source=chatgpt.com

Tras esta comparativa, concluimos que las herramientas OCR más populares son: PaddleOCR para alta precisión y soporte multi-idioma en un único modelo, Tesseract para bajo consumo y ejecución rápida en CPU y Surya para documentos complejos (tablas, matemáticas, etc.). Otras alternativas son Docling, Easy, Mistral, Qwen2.5-VL, InternVL y GOT-OCR 2.0.

Adicionalmente, se evaluó el interés que estas herramientas suscitan mundialmente en los últimos cinco años utilizando [Google Trends](#), que genera una puntuación entre el 0 y el 100

según cuántas búsquedas se hacen sobre cada algoritmo (Google Trends, s. f.). PaddleOCR y Tesseract son de lejos los más buscados. Aunque otros de los ya citados están aumentando su popularidad, se decidió experimentar con PaddleOCR y Tesseract dada su gran aceptación.

Figura 1 Evolución de la popularidad de los principales algoritmos de OCR (2021-2026)



Fuente: Google Trends para los datos, elaboración propia para la visualización.

[PaddleOCR es de código abierto, está desarrollado por Baidu \(Baidu, s. f.\) y puede](#) procesar múltiples idiomas y digitalizar documentos escritos a mano, entre otras funciones. Está entrenado con conjuntos de datos multilingües masivos, siguiendo las recomendaciones de la investigación de Biró et al. (2023). También se basa parcialmente en modelos genéricos, como proponen Lin et al. (2022), ya que <https://www.mdpi.com/2076-3417/13/24/13107https://arxiv.org/abs/2212.09297> utiliza ResNet o transformers (según la versión).

[Tesseract](#) fue desarrollado entre 1985 y 1994 por HP, se volvió de código abierto en 2005 (mantenido principalmente por Google) (tesseract-ocr, 2026). Se orienta a ser ejecutado en CPUs y a casos de uso sencillos. Utiliza un modelo distinto para cada idioma y una arquitectura LSTM propia.

La filosofía clásica de Tesseract lo hace ideal para casos sencillos que precisen de velocidad y ejecución en CPU. PaddleOCR, sin embargo, ofrece funciones modernas, soporte multilingüe

y mejor rendimiento. En la sección 5.2.2 se experimenta con ambos, aunque PaddleOCR parece adaptarse mejor a las necesidades del sistema, ya que requerimos de reconocimiento de texto en fotografías y el uso de recursos computacionales no es crítico.

2.2.3. *Dashboards*

En este contexto, el *dashboard* analítico no es un componente accesorio del sistema, sino la capa donde el valor del proyecto se hace visible y defendible. Un panel de control operativo bien diseñado permite que un responsable de mantenimiento, sin conocimientos estadísticos avanzados, pueda identificar qué activos concentran mayor tiempo de parada, qué familias de fallo están creciendo, qué procedimientos documentales se asocian con determinados síntomas, y qué escenarios de riesgo pueden anticiparse a partir del comportamiento histórico. La visualización, en este sentido, no es decorativa: es la interfaz entre el modelo de datos y la toma de decisiones.

2.2.3.1. Analítica visual aplicada al mantenimiento

La visualización analítica se ha convertido en un elemento clave para transformar los datos de mantenimiento en información útil para la toma de decisiones. En el contexto del mantenimiento y soporte técnico, estas herramientas permiten combinar modelos analíticos con representaciones visuales que facilitan la interpretación de los resultados por parte del personal técnico y de gestión.

El análisis realizado por Cheng et al. (2022) identifica cuatro grandes áreas de aplicación de la visualización analítica en mantenimiento y soporte técnico:

- Detección de anomalías en activos industriales.
- Planificación y programación de intervenciones de mantenimiento.
- Análisis exploratorio de datos (o EDA, por las siglas en inglés de *Exploratory Data Analysis*).
- Técnicas de inteligencia artificial explicable.

Uno de los hallazgos más relevantes de este estudio es la existencia de una brecha significativa entre los sistemas de predicción y las interfaces que presentan sus resultados. Muchos

trabajos académicos se centran en mejorar la precisión de los modelos predictivos, pero dedican menos atención a cómo estos resultados son interpretados por los usuarios finales. Como consecuencia, los sistemas desarrollados suelen ofrecer predicciones complejas sin proporcionar herramientas visuales que faciliten la comprensión de los problemas y su historial por parte del personal técnico y/o el personal de mantenimiento de las máquinas.

Además, se puede destacar que la participación de los profesionales de mantenimiento en el diseño y evaluación de estos sistemas sigue siendo limitada. En particular, la retroalimentación de los usuarios, tanto durante la construcción del modelo como en su uso operativo. Este aspecto refuerza la necesidad de diseñar plataformas analíticas centradas en el usuario, capaces de combinar análisis de datos con interfaces visuales intuitivas.

En conjunto, estos estudios muestran que, aunque los sistemas de mantenimiento y ayuda al soporte técnico han avanzado significativamente en el desarrollo de modelos analíticos, todavía existe un espacio relevante para mejorar la forma en que los datos y resultados se presentan a los usuarios.

2.2.3.2. Indicadores clave de rendimiento y criticidad de activos

El uso de indicadores clave de rendimiento (o KPIs, por las siglas en inglés de *Key Performance Indicators*) es una práctica consolidada en la ingeniería de mantenimiento y fiabilidad. Estos indicadores permiten evaluar el comportamiento de los activos y apoyar la toma de decisiones relacionadas con la planificación de intervenciones.

Entre los indicadores más utilizados tradicionalmente en la literatura destacan métricas operativas como el Tiempo Medio de Reparación (MTTR), orientado a medir la eficiencia ante averías, y la volumetría del Total de Incidencias registradas. No obstante, investigaciones recientes sugieren que el análisis basado únicamente en indicadores individuales y aislados puede resultar limitado en entornos complejos. Por este motivo, diversos trabajos proponen la combinación de múltiples dimensiones del mantenimiento, tales como la gravedad de los fallos, el Coste Total Estimado asociado a las intervenciones y las métricas de calidad o Estado de la Validación de los reportes, para priorizar los activos más problemáticos y asegurar la integridad de la infraestructura frente a Errores de Sistema.

En este sentido, el estudio de Hector y Panjanathan (2024) describe diferentes arquitecturas de mantenimiento predictivo en el contexto de la Industria 4.0. Estos autores identifican una serie de etapas comunes en los sistemas analíticos de mantenimiento, entre las que destacan:

- Limpieza y preparación de datos.
- Normalización y estructuración de la información.
- Extracción de características relevantes.
- Modelización y análisis de la criticidad de los activos.

Este enfoque coincide con la arquitectura analítica propuesta en el sistema OpsInsight Analytics, donde el análisis de incidencias se basa en un *pipeline* estructurado que transforma registros de mantenimiento originalmente heterogéneos en indicadores analíticos normalizados.

En el *dashboard* desarrollado en este proyecto, la criticidad de los activos no se limita a contabilizar de forma aislada el número de fallos. En su lugar, el sistema está diseñado para estructurar y ponderar un conjunto multidimensional de indicadores que componen el estado operativo real, organizados conceptualmente de la siguiente manera:

1. **Dimensión de carga y eficiencia:** Controlada a través del Total de Incidencias y el Tiempo Medio de Reparación (MTTR).
2. **Dimensión económica y de severidad:** Evaluada mediante el cálculo del Coste Total Estimado de los fallos y la categorización de los eventos según su Distribución por Criticidad (Baja, Media, Alta).
3. **Dimensión de fiabilidad y calidad del dato:** Monitorizada mediante la tasa de Reportes con Validación OK y la identificación de Alertas Semánticas Detectadas en los textos técnicos.

El resultado de estas dimensiones analíticas se representa de forma integrada mediante componentes interactivos que combinan paneles visuales de distribución y gráficos de criticidad. Asimismo, el sistema fundamenta la necesidad teórica de un acceso granular a la información a través de un registro de incidencias en modo resumen en formato tabular. Este consolida los datos relevantemente críticos de cada evento, identificador, fecha, máquina afectada, operario asignado, criticidad, tiempos, costes estimados, estado de validación y el diagnóstico automatizado, garantizando la trazabilidad de la auditoría. Esta estructura

conceptual permite filtrar y explotar la información de manera dinámica para realizar una gestión del mantenimiento basada rigurosamente en evidencias.

2.2.3.3. Herramientas de visualización: Grafana como *stack* principal

En el marco de la ingeniería de datos y el mantenimiento industrial, la selección del ecosistema de visualización determina no solo la estética del panel, sino la robustez de la toma de decisiones basada en evidencias. Un *dashboard* operativo bien diseñado actúa como la interfaz crítica entre el modelo de datos analítico y el personal técnico, permitiendo identificar activos que concentran paradas, familias de fallos crecientes y escenarios de riesgo (Cheng et al., 2022). Para el desarrollo de OpsInsight Analytics, se ha llevado a cabo una evaluación comparativa entre tres soluciones principales del estado del arte.

Las librerías de visualización interactiva como puede ser Plotly, se identifica en la literatura como una de las librerías más potentes para la generación de gráficos interactivos en el ecosistema Python. Su principal fortaleza reside en ofrecer funcionalidades avanzadas de exploración visual, como herramientas de *zoom*, filtrado dinámico en el cliente y visualización de metadatos detallados mediante *tooltips*. Estas características la convierten en una herramienta idónea para construir visualizaciones dinámicas y detalladas que requieren un alto grado de interactividad manual. Sin embargo, como componente individual, Plotly carece de una capa de gestión de interfaz de usuario, obligando al uso de *frameworks* adicionales (como Dash o Flask) para construir la aplicación web, lo que incrementa significativamente la complejidad del desarrollo *frontend* y el mantenimiento de la arquitectura.

Inicialmente, se consideraron otras opciones como Streamlit como el eje central de la capa de presentación. Streamlit es una herramienta de código abierto que permite construir aplicaciones web interactivas directamente desde Python, eliminando la necesidad de conocimientos avanzados en desarrollo web tradicional. Según Taoufyq et al. (2025), Streamlit destaca por su simplicidad y su capacidad para integrar de forma fluida modelos de lenguaje de gran escala (LLM) y lógica de negocio en un único entorno, lo que facilita el desarrollo rápido de estructuras analíticas completas.

No obstante, durante la fase de integración del sistema, se identificaron limitaciones estructurales en Streamlit para un entorno operativo industrial continuo:

1. Gestión de persistencia y concurrencia: La gestión de estados de sesión y la conexión permanente a bases de datos relacionales para múltiples usuarios requiere una lógica de servidor personalizada que puede comprometer la estabilidad bajo alta carga.
2. Despliegue y mantenimiento: Streamlit carece de mecanismos nativos de auto-provisionamiento industrial, lo que obliga a configuraciones manuales de los componentes visuales en cada nuevo despliegue del sistema.

Ante la necesidad de una solución orientada a la operación continua y no solo al análisis estático, se ha optado por la transición hacia Grafana 11.4. Esta plataforma se ha consolidado como el estándar de facto para la observabilidad y monitorización de infraestructuras en tiempo real (Raza et al., 2023). La selección de Grafana como la capa de visualización definitiva de *OpsInsight Analytics* se sustenta en las siguientes ventajas competitivas detectadas.

1. Conectividad Nativa y Desacoplamiento: Al utilizar MySQL 8.0 como repositorio central, Grafana permite consultas directas mediante SQL estándar. Esto elimina el acoplamiento entre la lógica de negocio del agente IA y la capa de presentación, asegurando que KPIs críticos como el MTTR y el coste total estimado se actualicen automáticamente con cada nueva inserción en la base de datos.
2. Auto-provisionamiento (*Dashboards-as-Code*): Una de las aportaciones técnicas clave de este proyecto es la capacidad de despliegue automatizado. Mediante el uso de archivos de configuración .yaml y .json integrados en el entorno Docker, el *dashboard* se aprovisiona automáticamente al arrancar el sistema. Esto garantiza la reproducibilidad total: cualquier técnico que levante el entorno obtendrá el mismo panel configurado sin necesidad de intervención manual.
3. Gestión Integral de Errores e Infraestructura: A diferencia de las herramientas de ciencia de datos puras, Grafana permite unificar en un único panel las métricas de negocio (incidencias reportadas) y las métricas de salud del sistema. Esto ha permitido implementar el panel de "Errores de Sistema", donde se visualizan los fallos capturados por el *ErrorHandler* global (criticidad SISTEMA_ERROR), permitiendo una trazabilidad técnica inmediata ante degradaciones del servicio o fallos de los LLMs locales.
4. Representación de Alertas Semánticas: La flexibilidad de los paneles de Grafana facilita la representación visual de las inconsistencias detectadas por el módulo de validación

(ej. LPS_INCOHERENTE o COMP_DV_ELECTRIC), transformando las reglas del manual técnico MetroPT-3 en indicadores visuales de calidad del dato de forma inmediata para el operario.

En conclusión, mientras que Plotly y Streamlit representan soluciones óptimas para el análisis exploratorio y la investigación académica, la naturaleza del sistema *OpsInsight Analytics* como herramienta de soporte a la decisión en tiempo real motivó la elección de Grafana. Esta decisión estratégica garantiza una infraestructura escalable, profesional y alineada con los estándares actuales de monitorización en la Industria 4.0.

2.3. ANÁLISIS DE DATOS DE MANTENIMIENTO E INCIDENCIAS

El cuarto pilar del estado del arte del presente proyecto lo constituye la revisión de los *datasets* y metodologías de análisis de incidencias y fallos en entornos operativos e industriales. A diferencia de las secciones anteriores —centradas en las tecnologías de extracción y visualización—, este bloque aborda el problema desde la perspectiva del dato: qué información registran los sistemas de mantenimiento, en qué formatos se almacena, qué limitaciones estructurales presentan y cómo la literatura especializada ha abordado su explotación analítica. Esta revisión resulta esencial porque determina el diseño del modelo de datos de OpsInsight Analytics y justifica las decisiones metodológicas adoptadas en la fase de análisis.

2.3.1. Datasets de referencia en mantenimiento

El AI4I 2020 Predictive Maintenance Dataset (Matzka, 2020), disponible en el UCI Machine Learning Repository bajo licencia CC BY 4.0, es un conjunto de datos sintético diseñado para simular el comportamiento de equipos industriales. Contiene 10.000 registros con variables de proceso —temperatura del aire, temperatura del proceso, velocidad rotacional, par y desgaste de herramienta— y cinco tipos de fallo etiquetados (fallo de herramienta, fallo de disipación de calor, fallo de potencia, sobrecarga y fallo aleatorio). Su naturaleza sintética garantiza la ausencia de datos personales y permite trabajar con un esquema de fallos bien definido, lo que lo convierte en el *dataset* de referencia principal para validar el módulo de análisis de criticidad y frecuencia de fallos por activo en OpsInsight Analytics.

El MetroPT-3 Dataset (Veloso et al., 2022), también disponible en el *UCI Machine Learning Repository*, complementa al anterior con datos reales de sensores de un compresor de aire del sistema de metro de Oporto. Contiene series temporales de alta frecuencia de presiones, temperaturas y flujos de aceite, con anotaciones de fallos identificados por técnicos de mantenimiento. Su resolución temporal y la naturaleza real de los datos —frente a la generación sintética del AI4I 2020— permiten trabajar con series temporales de degradación y modelizar el comportamiento previo al fallo. Para OpsInsight Analytics, este *dataset* es especialmente relevante en el bloque de análisis temporal y en la validación de la capacidad de la plataforma para predecir necesidades de mantenimiento sobre datos de naturaleza no tabular.

Esta base de datos es la que sirvió como referencia para generar reportes de error para validar el sistema, así como para generar un manual técnico sintético. Todo ello está basado en la documentación de este *dataset* así como en las variables monitorizadas en los registros del mismo.

Adicionalmente, se incluye la siguiente estimación de costes de intervención en la documentación utilizada por los agentes para que sean capaces de estimar el gasto necesario para resolver los problemas reportados:

Tabla 1 Estimación de costes utilizada por los agentes.

Concepto	Coste	Notas
Mano de Obra Técnica	45 €/hora	Aplicable a técnicos internos.
Especialista Externo	90 €/hora	Requerido para fallos en motor o APU.
Kit de Filtros y Aceite	120 €	Material fungible básico.
Solenoides/Válvula Repuesto	350 €	Coste medio de componentes neumáticos.

Parada de Línea No Programada	500 €/hora	Coste de oportunidad por indisponibilidad.
-------------------------------	------------	--

Fuente: Elaboración propia

2.3.2. Limitaciones estructurales de los datos operativos reales

Una de las principales dificultades que plantea el análisis de incidencias en entornos industriales reales no es la ausencia de datos, sino su heterogeneidad estructural. Galar et al. (2015) documentan cómo en plantas industriales típicas la información de mantenimiento se distribuye entre sistemas de gestión de activos (CMMS/EAM), PDFs, partes en papel y formularios, lo que impide construir series temporales completas y homogéneas sin un proceso previo de integración y limpieza. Esta observación está en la base del problema que OpsInsight Analytics intenta resolver: antes de analizar, es necesario estructurar.

En esa misma línea, Mobley (2002) señala que incluso cuando los datos están disponibles en formato digital, su calidad analítica es frecuentemente limitada por inconsistencias de nomenclatura, campos incompletos, duplicados y ausencia de categorías de fallo normalizadas. La consecuencia directa es que un porcentaje significativo del tiempo en proyectos de ayuda al mantenimiento y al soporte técnico (especialmente si son predictivos) se dedica a la preparación del dato. Esta constatación justifica que en el presente proyecto el agente normalizador ocupe una posición central en la arquitectura, y que el diseño del modelo de datos analítico se trate como una aportación técnica en sí misma, no como un paso previo menor.

2.3.3. Metodologías de análisis de fallos e incidencias en la literatura

La literatura sobre análisis de incidencias operativas se ha desarrollado principalmente desde dos tradiciones metodológicas. La primera, procedente de la ingeniería de fiabilidad, se apoya en técnicas como el análisis modal de fallos y efectos (AMFE/FMEA), el árbol de fallos (FTA) y el análisis de causa raíz (RCA), que permiten identificar los modos de fallo de un activo y sus consecuencias potenciales (Stamatis, 2003). Estas técnicas, aunque útiles para el diseño de

sistemas de mantenimiento preventivo, son predominantemente cualitativas y requieren un conocimiento experto del sistema que no siempre está disponible.

La segunda tradición, de naturaleza cuantitativa, utiliza métodos estadísticos y de aprendizaje automático sobre datos históricos de operación para detectar anomalías, clasificar tipos de fallo y predecir su ocurrencia. Raza et al. (2023) presentan una revisión sistemática sobre el estado del arte en detección de anomalías en series temporales industriales, identificando como técnicas más utilizadas los modelos de regresión con variables de proceso, los métodos basados en *clustering* (K-Means, DBSCAN), los árboles de decisión y los modelos de series temporales (ARIMA, LSTM). La mayor parte de los trabajos operan sobre datos ya estructurados y limpios, sin abordar el problema de la integración de fuentes heterogéneas que sí constituye el núcleo del presente proyecto.

De especial relevancia para el diseño del índice de criticidad de OpsInsight Analytics es la propuesta de Haddad et al. (2012), que desarrollan un marco de priorización de activos basado en un indicador compuesto que combina frecuencia de fallo, tiempo de reparación y coste directo. Su validación sobre datos reales de una planta de manufactura muestra que este tipo de índice proporciona una base más robusta para planificar el mantenimiento que los enfoques basados en un solo indicador. Este trabajo sustenta directamente la decisión de diseñar un índice de criticidad multidimensional en la capa analítica de la plataforma, en lugar de limitarse a contabilizar el número de incidencias por activo.

2.4. Conclusiones

En relación con el análisis de datos de mantenimiento e incidencias, la revisión de los principales *datasets* de referencia —AI4I 2020, MetroPT-3 y el NASA Prognostics Data Repository— pone de manifiesto que existen bases de datos bien documentadas y con licencias que permiten su uso académico, lo que elimina la dependencia de datos reales de empresa para construir el repositorio analítico inicial. Al mismo tiempo, la literatura evidencia que el problema no está en la disponibilidad de datos, sino en su heterogeneidad estructural: registros distribuidos en múltiples fuentes, nomenclaturas inconsistentes y campos incompletos obligan a diseñar un pipeline de integración cuidadoso antes de poder aplicar cualquier técnica analítica. Esta constatación refuerza la pertinencia del enfoque adoptado en

OpsInsight Analytics, donde el agente normalizador y el diseño del modelo de datos analítico se conciben como aportaciones técnicas nucleares, no como pasos preparatorios menores.

En cuanto a los LLMs locales y los sistemas multiagentes, el estado del arte muestra que los modelos de lenguaje de rango medio-bajo —con 3B parámetros, ejecutables en CPU con pocos *gigabytes* de RAM mediante herramientas como Ollama— son ya suficientemente capaces para tareas de extracción estructurada de información técnica. Esto abre la puerta a pipelines de análisis documental completamente locales, sin dependencias de servicios en la nube ni exposición de datos sensibles. Sin embargo, ninguna de las soluciones estudiadas integra esta capacidad de extracción con un repositorio analítico homogéneo ni con una capa de visualización operativa. La arquitectura multiagente propuesta en OpsInsight Analytics — con agentes diferenciados de ingesta, extracción, normalización, consulta documental y análisis— representa una respuesta directa a este vacío, combinando la flexibilidad de los LLMs locales con la trazabilidad y reproducibilidad que exige un entorno de mantenimiento industrial.

En lo relativo a la visualización y los *dashboards*, la revisión de la literatura pone de manifiesto que la mayoría de los trabajos sobre mantenimiento predictivo se centran en mejorar la precisión de los modelos analíticos, pero dedican escasa atención a cómo presentar esos resultados de forma comprensible para el personal técnico o administrativo sin formación estadística avanzada para que puedan actuar en base a la información recopilada. Esta brecha es precisamente el espacio que ocupa la capa de visualización de OpsInsight Analytics. Para que un *dashboard* aporte valor real, es imprescindible que los datos subyacentes estén bien estructurados y normalizados; la utilidad de cualquier visualización depende directamente de la calidad del modelo de datos que la alimenta.

Finalmente, Grafana se presenta como la herramienta más adecuada para el alcance de este proyecto: permite construir aplicaciones escalables, profesionales y alineadas con los estándares actuales de monitorización. Otras alternativas como Plotly o Streamlit, aunque también tienen un gran potencial, se orientan más al análisis exploratorio. También se consideró Kibana, pero Grafana se adapta mejor a potenciales fuentes de datos adicionales.

3. Objetivos concretos y metodología de trabajo

Este apartado presenta los objetivos del proyecto, comenzando por los generales (sección 3.1) para luego profundizar en los específicos (sección 3.2). También se divide el proceso de ejecución del proyecto en una serie de fases descritas en la sección 3.3.

3.1. Objetivo general

El presente trabajo tiene como objetivo general diseñar, implementar y validar OpsInsight Analytics, una plataforma multiagente local capaz de transformar información operativa dispersa y heterogénea —registros de incidencias en formatos no estructurados, documentación técnica y manuales de operación— en una base de conocimiento analítico estructurada, trazable y visualmente explotable, orientada al análisis de incidencias en entornos industriales de operación y mantenimiento.

La plataforma integra tres líneas técnicas complementarias: extracción y estructuración automática de documentación operativa mediante OCR y modelos de lenguaje locales; validación del dato y construcción de un repositorio analítico sobre el que aplicar análisis descriptivo de incidencias; y visualización interactiva que permita identificar patrones de fallo y estimar la criticidad y los costes de las averías. El resultado buscado es la demostración rigurosa de que es posible construir una herramienta de soporte a la decisión íntegramente local, cuyo núcleo de valor es la orquestación automatizada del flujo de datos. Esto incluye la capacidad de dotar de estructura a lo no estructurado y de fundamentar cada diagnóstico en evidencias documentales, cerrando así la brecha entre la documentación técnica estática y la operación diaria en entornos industriales.

3.2. Objetivos específicos

OE1. Analizar el dominio del mantenimiento industrial y las características de los conjuntos de datos AI4I 2020 y MetroPT-3 para establecer las variables analíticas relevantes, los tipos de incidencia presentes y las limitaciones de formato que requieren tratamiento documental previo.

OE2. Realizar un análisis comparativo de las soluciones OCR disponibles —Tesseract y PaddleOCR — evaluando su precisión, requisitos computacionales y adecuación al procesamiento de documentación técnica industrial en entornos con restricciones de hardware, con el objetivo de seleccionar justificadamente la herramienta más apropiada para el pipeline de extracción del proyecto.

OE3. Implementar un módulo de ingesta y extracción documental capaz de procesar formularios *online*, partes de incidencia en PDF, fotografías de reportes (impresos o manuscritos) aplicando OCR y modelos de lenguaje locales para obtener registros en el esquema JSON definido con un nivel de completitud evaluado mediante métricas de extracción.

OE4. Desarrollar un agente de normalización semántica que unifique nomenclaturas de activos, resuelva variantes terminológicas en los tipos de fallo y relacione cada incidencia con los fragmentos más relevantes del corpus de manuales técnicos mediante recuperación basada en similitud semántica.

OE5. Construir un repositorio analítico consolidado sobre MySQL que integre los registros estructurados, metadatos documentales y campos de validación, permitiendo consultas analíticas eficientes sobre el histórico completo de incidencias sin dependencia de servicios externos.

OE6. Realizar un análisis descriptivo y temporal del repositorio que identifique los tipos de incidencia más frecuentes, los activos con mayor tiempo de parada acumulado, los patrones estacionales o cíclicos y las correlaciones significativas entre variables operativas.

OE7. Diseñar e implementar un índice de criticidad de activos que combine frecuencia de fallo, duración media de parada y coste directo estimado, proporcionando una métrica de priorización operativa trazable y justificable metodológicamente.

OE8. Desarrollar un sistema de diagnóstico técnico inteligente que, mediante técnicas de RAG, identifique soluciones y estime KPIs operativos (costes y tiempos) basados en manuales técnicos.

OE9. Construir un *dashboard* interactivo en Grafana que permita la monitorización operativa y el seguimiento de errores de sistema, consultar procedimientos correctivos asociados y trazar cada registro hasta su documento fuente original.

3.3. Metodología del trabajo

Este apartado distingue entre el marco metodológico adoptado para el desarrollo del TFM — la lógica de progresión entre fases y los criterios de evaluación— y el plan técnico de implementación, que concreta los módulos, herramientas y arquitectura del sistema.

3.3.1. Marco metodológico: CRISP-DM

Como marco de referencia para la gestión del proceso de análisis de datos, el proyecto adopta CRISP-DM (*Cross-Industry Standard Process for Data Mining*), estándar ampliamente reconocido en proyectos de analítica de datos e inteligencia artificial. CRISP-DM estructura el proceso en seis fases iterativas —comprensión del dominio, comprensión de los datos, preparación de los datos, modelado, evaluación y despliegue— que se corresponden directamente con las fases de desarrollo de OpsInsight Analytics. Su carácter cíclico permite que los hallazgos en fases avanzadas retroalimenten decisiones tomadas en etapas previas, especialmente útil dado que el diseño del modelo de datos y el pipeline de extracción evolucionarán conforme se comprenda mejor el corpus de incidencias.

La evaluación del proyecto se articulará en torno a tres dimensiones: (1) calidad de extracción, medida mediante métricas de completitud y precisión (F1) de los campos extraídos por el módulo OCR/LLM; (2) calidad analítica y coherencia del esquema normalizado; y (3) usabilidad del *dashboard*, valorada mediante una prueba con usuarios del perfil técnico objetivo.

La correspondencia entre fases CRISP-DM, tareas, responsables y entregables se recoge en la siguiente tabla.

Tabla 2 Fases del proceso CRISP-DM

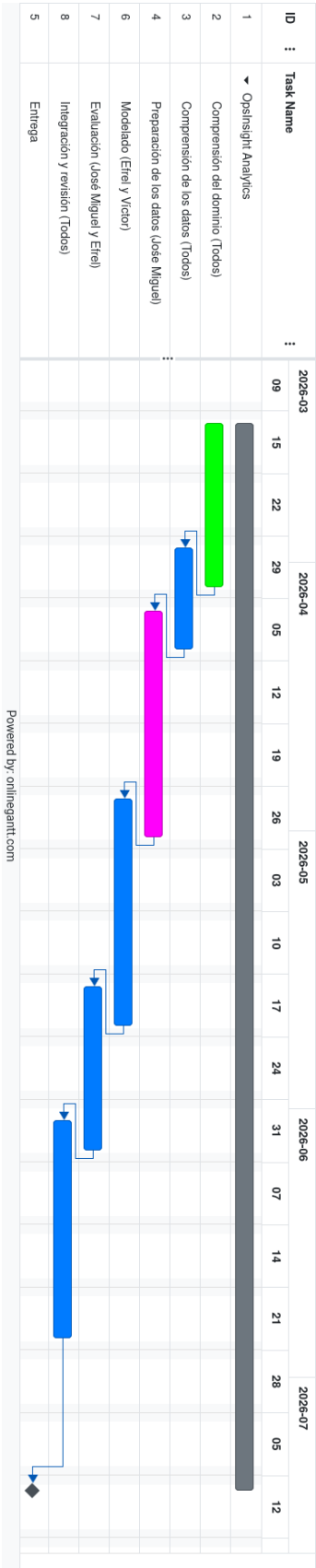
Fase CRISP-DM	Tareas principales	Responsable	Semanas	Entregable
1. Comprensión del dominio	Análisis de <i>datasets</i> (AI4I 2020, MetroPT-3). Definición de variables analíticas y tipos de incidencia. Revisión de manuales técnicos de referencia.	Todos	1-3	Informe de dominio y OE1
2. Comprensión de los datos	Exploración estadística del AI4I 2020. Identificación de inconsistencias, valores ausentes y distribución de tipos de fallo. Análisis de calidad del corpus documental.	Todos	3-4	Manual sintético, plantilla de reporte de fallo y OE1
3. Preparación de los datos	Pipeline OCR (Tesseract y PaddleOCR). Agente extractor y generación de incidencias sintéticas.	José Miguel	4-7	Módulos OE3, OE4 y OE5
4. Modelado	Análisis descriptivo y temporal (OE6). Índice de criticidad compuesto (OE7). Diseño e implementación del modelo de datos en MySQL.	Efrel y Víctor	7-10	Módulos OE6, OE7 y OE8

5. Evaluación	Evaluación del OCR con varias entradas, de la coherencia del esquema normalizado y de la usabilidad del <i>dashboard</i> .	José Miguel (OCR) y Efrel (dashboard)	10-12	Módulos OE2 y OE9
6. Integración, revisión y entrega	Integración de la arquitectura multiagente completa. Aprovisionamiento automático de <i>dashboard</i> en Grafana mediante Docker. Documentación técnica y entrega del TFM.	Todos	12-16	Plataforma + TFM final

Fuente: Elaboración propia

A continuación, se muestra el diagrama de Gantt que hicimos para organizar el desarrollo del proyecto y visualizar las fechas límites de cada etapa. Tal y como puede apreciarse, hay una semana de solape entre cada tarea, ya que normalmente alguno de nosotros acababa sus tareas antes del resto y podía comenzar con las siguientes. Además, dejamos dos semanas al final como *buffer* por si acaso surgían imprevistos, aunque no fue el caso y pudimos terminar el trabajo con margen antes de la fecha límite para la entrega.

Figura 2 Diagrama Gantt con las tareas planificadas



Fuente: Elaboración propia (con onlinegantt.com)

3.3.2. Plan técnico de implementación

El sistema se construye sobre una arquitectura de dos agentes especializados (detallada en la sección 5.2), orquestados mediante n8n. Las herramientas seleccionadas son: Python como lenguaje principal, Tesseract o PaddleOCR como motor de extracción de texto a partir de imágenes, modelos LLM locales ejecutados mediante Ollama, MySQL como repositorio analítico y Grafana para la visualización.

Para delimitar el alcance real en el contexto de este TFM, los componentes se clasifican según su nivel de implementación previsto:

- **Componentes nucleares** (implementación completa y validada): Módulo de ingesta documental y extracción OCR/LLM, registro de incidencias en MySQL y *dashboard* en Grafana.
- **Componentes de alcance prototipo**: Agente analista con recuperación semántica de manuales (funcional para el corpus de pruebas, no generalizable sin ajuste adicional).

La distribución de responsabilidades entre los miembros del equipo se recoge en la Tabla 2.

Tabla 3 Distribución de responsabilidades por miembro del equipo

Tarea / Entregable	Efrel Armando	José Miguel	Víctor
Estado del arte: LLMs y agentes (§2.2.1)	✓		
Estado del arte: OCR (§2.2.2)		✓	
Estado del arte: Dashboards (§2.2.3)			✓
Objetivos y metodología (cap. 3)	✓	✓	✓
Marco normativo (cap. 4)	✓		
Arquitectura de agentes (§5.1)		✓	✓
Validación del JSON, control de errores y monitorización (§5.x)	✓		
Desarrollo OCR e ingesta (§5.x)		✓	
Desarrollo dashboard (§5.x)	✓		

Integración y pruebas (§5.x)	✓	✓	✓
Conclusiones y prospectiva (cap. 7–8)	✓	✓	✓

Fuente: Elaboración propia

4. Marco normativo

Este capítulo analiza el marco regulatorio y las condiciones de uso aplicables a los datos, modelos y herramientas de OpsInsight Analytics. El análisis cubre cuatro bloques: protección de datos personales, licencias de los conjuntos de datos, licencias de los modelos de inteligencia artificial, y consideraciones éticas.

4.1.PROTECCIÓN DE DATOS PERSONALES

El Reglamento General de Protección de Datos (RGPD, Reglamento UE 2016/679) y la Ley Orgánica 3/2018 de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD) establecen el marco jurídico aplicable al tratamiento de datos personales en la Unión Europea.

En el contexto de OpsInsight Analytics, el tratamiento de datos personales puede producirse de forma indirecta si los documentos operativos procesados —partes de incidencia, órdenes de trabajo, registros de actuación— contienen datos identificativos de trabajadores (nombre del técnico). El sistema no está diseñado para tratar datos personales como fin principal, sino para analizar el comportamiento de los activos. No obstante, dado que estos datos pueden estar presentes en los documentos de entrada, aplica el principio de minimización del dato (art. 5.1.c RGPD): El agente normalizador debe identificar y descartar o pseudoanonimizar los campos identificativos que no sean necesarios para el análisis de la incidencia antes de su incorporación al repositorio analítico.

La base jurídica del tratamiento, en el caso de un despliegue real en el entorno de Sopra Steria, sería el interés legítimo del responsable del tratamiento (art. 6.1.f RGPD) en la optimización de sus procesos de mantenimiento, siempre que este interés no prevalezca sobre los derechos y libertades de los interesados. Alternativamente, si los datos de empleados se tratan de forma sistemática, resultaría aplicable la base de ejecución de un contrato laboral (art. 6.1.b RGPD). En cualquier caso, el responsable del tratamiento deberá llevar a cabo un análisis de intereses legítimos y, si el tratamiento implica evaluación sistemática de personas, una Evaluación de Impacto en la Protección de Datos (EIPD) conforme al art. 35 RGPD.

En el entorno académico y de validación del presente TFM, los conjuntos de datos utilizados son de naturaleza sintética (inspirados por los *datasets* AI4I 2020 y MetroPT-3, pero totalmente creados por nosotros) y no contienen datos de personas físicas identificadas o identificables, por lo que no resulta de aplicación el RGPD a efectos de este trabajo de investigación. La documentación sintética generada ad hoc para las pruebas de OCR y extracción tampoco incluye datos personales reales.

4.2.LICENCIAS DE LOS CONJUNTOS DE DATOS

Los conjuntos de datos considerados se distribuyen bajo licencias abiertas compatibles con uso académico y de investigación. AI4I 2020 Predictive Maintenance Dataset (UCI ML Repository): disponible bajo licencia CC BY 4.0, generado sintéticamente, sin datos personales. MetroPT-3 Dataset (UCI ML Repository): licencia CC BY 4.0, datos de sensores del metro de Oporto, sin información personal. Manuales técnicos de fuentes públicas (ManualsLib, Internet Archive, portales de Siemens, ABB, Schneider Electric): uso académico no comercial, empleados exclusivamente para recuperación semántica.

4.3.LICENCIAS DE LOS MODELOS DE INTELIGENCIA ARTIFICIAL

Los modelos integrados se distribuyen bajo licencias Apache 2.0 o MIT que permiten despliegue local sin restricciones: Qwen2.5 (Alibaba Cloud, Apache 2.0), Mistral Small 3 (Apache 2.0), Tesseract OCR (Apache 2.0 desde 2005), PaddleOCR (Apache 2.0), Grafana (AGPL v3 para la edición Open Source) y MySQL (GPL v2 para la Community Edition).

4.4.CONSIDERACIONES ÉTICAS

OpsInsight Analytics es una herramienta de soporte a la decisión, no un sistema de control autónomo. Todas las intervenciones de mantenimiento siguen siendo responsabilidad del personal técnico humano, que cuenta con las evidencias documentales generadas por la plataforma para fundamentar sus decisiones. El AI Act (Reglamento UE 2024/1689), en vigor desde agosto de 2024, encuadra este tipo de sistema en la categoría de riesgo limitado —sin capacidad de control directo sobre equipos críticos— y le aplica principalmente el principio de

transparencia. OpsInsight Analytics incorpora trazabilidad documental completa desde cada indicador hasta el registro de incidencia y el fragmento de manual técnico de origen.

5. Desarrollo específico de la contribución

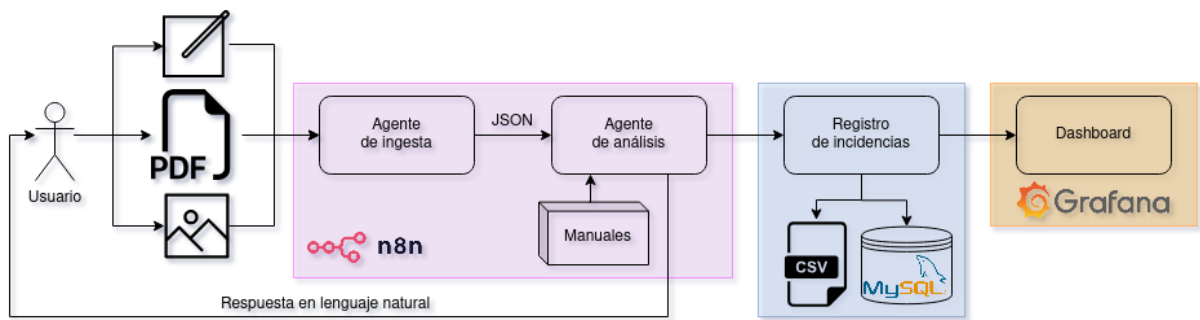
En este bloque se describe la contribución realizada, comenzando por una descripción del sistema para luego detallar sus componentes técnicamente en las subsecciones dedicadas.

OpsInsight Analytics es un sistema capaz de ingerir reportes de fallos en máquinas y proponer acciones a realizar para resolver dichos fallos. Se basa en agentes de IA que entienden lenguaje natural y ofrece un *dashboard* desde el que inspeccionar y analizar los reportes recibidos. El usuario objetivo es un operario o responsable de un entorno de producción. El sistema está compuesto por las siguientes partes:

- Un agente de ingesta de reportes capaz de procesar tanto documentos digitales como formularios manuscritos. Los reportes deben incluir el modelo de la máquina problemática, el error o motivo de la preocupación (ruido, vibración, olor, humo, fugas de líquidos o vapores, etc.) y los valores de las variables monitorizables (presión, temperatura, velocidad, etc.).
- Una etapa que valida el JSON generado por el módulo de ingesta. Si todo es correcto (formato, tipos, rangos de valores, etc.), el proceso seguirá con normalidad. Si no, se generará una excepción que se gestiona registrando el error en la base de datos con el registro de incidencias.
- Otro agente de IA que reciba estas incidencias en formato JSON y consulte los manuales de la/las máquinas para proponer una solución. Estos manuales deben incluir una sección de *troubleshooting* o solución de problemas, idealmente especificando los rangos de valores deseables para cada variable monitorizada.
- Un registro de incidencias que almacene en archivos CSV y una base de datos relacional (MySQL) las incidencias reportadas.
- Un *dashboard* con Grafana que permita inspeccionar el registro de incidencias, y analizar KPIs y visualizaciones relevantes.

A alto nivel, la arquitectura de la imagen representa el funcionamiento del sistema:

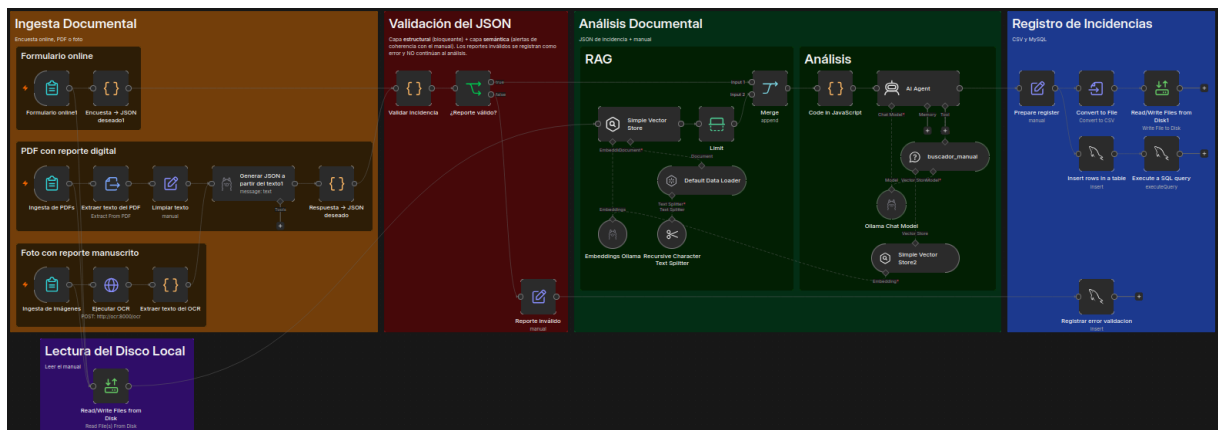
Figura 3 Esquema del funcionamiento del sistema y herramientas



Fuente: Elaboración propia

Adicionalmente, en esta otra imagen se muestra una visión más técnica de los componentes utilizados en n8n para implementar el sistema:

Figura 4 Vista general del workflow OpsInsight en n8n



Fuente: Elaboración propia

5.1. INFRAESTRUCTURA Y DESPLIEGUE DEL SISTEMA

La totalidad de OpsInsight Analytics se ejecuta de forma local mediante Docker Compose, que orquesta cinco servicios contenedorizados e independientes mediante una única definición declarativa (`docker-compose.yaml`). Esta decisión arquitectónica persigue tres objetivos: reproducibilidad total (cualquier equipo levanta el sistema completo con una sola orden), aislamiento de dependencias (cada componente fija su propia versión y entorno) y ausencia

de dependencias de servicios en la nube, requisito esencial dado el tratamiento de documentación operativa potencialmente sensible.

Los cinco servicios que componen el sistema se resumen en la siguiente tabla:

Tabla 4 Servicios contenedorizados que componen OpsInsight Analytics

Servicio	Imagen	Puerto	Función en el sistema
db	mysql:8.0	3306	Repositorio relacional de incidencias y errores (base opsinsight_db)
ollama	ollama/ollama	11434	Motor de modelos de lenguaje locales (extracción JSON, diagnóstico RAG y embeddings)
ocr	./ocr/paddle (build)	8001	API de OCR (PaddleOCR) que extrae texto de las fotografías de reportes
n8n	n8nio/n8n	5678	Orquestador de agentes: ejecuta el workflow y el manejador de errores
grafana	grafana/grafana:11.4.0	3000	Cuadro de mando de monitorización, auto-provisionado contra MySQL

Fuente: Elaboración propia

Los servicios se comunican entre sí a través de la **red interna** que Docker Compose crea automáticamente, resolviéndose por el nombre del servicio. Por ello, dentro del *workflow* las

llamadas se dirigen a `http://ollama:11434` y `http://ocr:8000` en lugar de a `localhost`: un contenedor que invocase `localhost` se referiría a sí mismo y no al servicio destino. Esta resolución por nombre desacopla los componentes y permite sustituir cualquiera de ellos sin reconfigurar el resto.

La **persistencia** se garantiza mediante volúmenes de Docker independientes del ciclo de vida de los contenedores: `mysql_data` conserva el histórico de incidencias, `n8n_data` conserva los *workflows* y credenciales, `ollama` conserva los modelos descargados y `grafana_data` conserva la configuración del panel. Adicionalmente, el servicio `n8n` monta dos carpetas del proyecto: `./datos` (manuales y ficheros de entrada) y `./tmp` (intercambio de ficheros). De este modo, detener o recrear los contenedores no implica pérdida de información.

El **orden de arranque** se controla con la directiva `depends_on`: tanto `n8n` como `grafana` declaran su dependencia de la base de datos y de los servicios de inferencia, de modo que no se inician hasta que sus dependencias están disponibles. El sistema completo se levanta con una sola orden:

```
docker compose up -d
```

Tras ella, el *workflow* queda accesible en `localhost:5678` y el *dashboard* en `localhost:3000`, este último ya provisionado con su fuente de datos y sus paneles sin intervención manual, como se detalla en la sección del *dashboard*.

5.2.ARQUITECTURA BASADA EN AGENTES

Para llevar a cabo las tareas expuestas en el planteamiento del problema, se propone una arquitectura basada en los siguientes agentes:

- **Agente extractor:** Recibe los reportes de fallos y extrae su información clave. Este agente trata cada posible entrada (formulario online, PDF digital o foto de un formulario rellenado a mano) de forma específica para transcribirlo a un documento JSON con una estructura definida y estandarizada.
- **Agente analista:** Dada la información de una incidencia en formato JSON, consulta los manuales relevantes a la misma para encontrar evidencias y procedimientos relevantes para sugerirle una acción al usuario.

Por ejemplo, si un manual especifica que la temperatura debe estar entre 50 y 70 °C y la presión debe de estar entre 3 y 3.5 atmósferas y se reporta una fuga de vapor a 60 °C y 4 atm, el sistema podría sugerir al operario que disminuya la temperatura de la máquina para intentar reducir la presión, ya que esta última está fuera de los valores recomendados mientras que la temperatura puede disminuirse y seguir en su rango deseable.

Si un manual especifica que una situación es crítica o peligrosa, el sistema sugiere una parada de emergencia y alertar al responsable de producción y a un técnico.

5.2.1. Agente de extracción documental

Este agente permite tres modos de entrada, todos ellos a partir de formularios online generados con n8n. Estos pueden ser rellenados ejecutando los *workflows* n8n desde su interfaz gráfica o desde las URLs que genera esta herramienta. Cabe destacar que se especifican los tipos de archivos que pueden ser subidos en cada tipo de formulario a través de una lista de extensiones válidas (.pdf y .PDF para los formularios digitales y .jpg, .JPG, .png, .PNG, .jpeg y .JPEG para las fotos de formularios manuscritos).

Tabla 5 Formatos de entrada

Formulario online	Reporte de Incidencia Indicar las presiones en bares (bar), la corriente en amperios (A), la temperatura en grados centígrados (°C). Para las señales eléctricas, marcar la casilla si está encendida y dejar sin marcar si está apagada. Fecha * mm / dd / yyyy Máquina problemática * MetroPT3 Nombre del operario *
Formulario PDF	Reportar una Incidencia con un PDF Formulario para reportar incidencias mediante reportes digitales en formato PDF. Archivo PDF * Browse... No files selected. Submit Form automated with  n8n

Foto de un manuscrito	Reportar una Incidencia con un Manuscrito Formulario para reportar incidencias mediante imágenes de reportes rellenados a mano. Fotografía del reporte * Browse... No files selected. Submit Form automated with  n8n
-----------------------	---

Fuente: Elaboración propia

El objetivo final de este componente es generar un JSON con el siguiente formato:

Tabla 6 Ejemplo de incidencia en formato JSON

Ejemplo de JSON

```
{
  "fecha": "2026-05-18",
  "maquina": "MetroPT3",
  "operario": "José Miguel Torres Cámara",
  "descripcion": "La máquina hace un ruido extraño.",
  "variables": {
    "presiones": {
      "tp2": 12.4,
      "tp3": 10.9,
      "hl": 3.6,
      "dv": 0.0,
      "r": 11.1
    }
  }
}
```

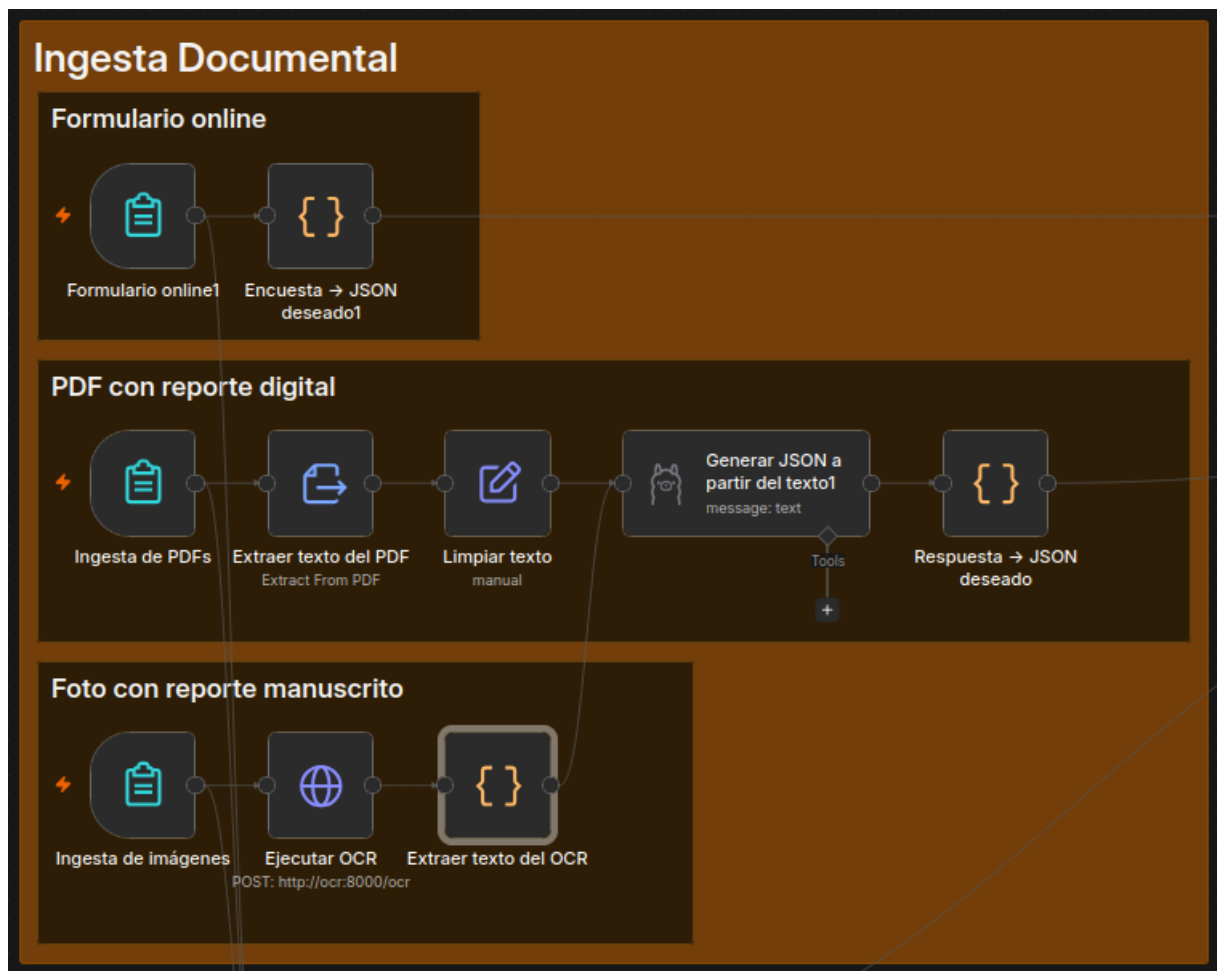


```
},  
  "senales_electricas": {  
    "comp": 0,  
    "dv_electric": 1,  
    "towers": 1,  
    "mpg": 0,  
    "lps": 0,  
    "pressure_switch": 0,  
    "oil_level": 0  
  },  
  "corriente_motor": 6.8,  
  "temperatura_aceite": 63  
}  
}
```

Fuente: Elaboración propia

Al recibir una entrada por cualquiera de las fuentes mencionadas, se sigue un proceso específico al tipo de entrada recibido, tal y como se aprecia en la imagen.

Figura 5 Módulo de ingesta documental en n8n



Fuente: Elaboración propia

El formulario online permite especificar los tipos de las variables a introducir, así como si son o no obligatorias, por lo que permite asegurar que se rellenen todos los campos con los tipos correctos (fechas, números, texto, etc.). Como particularidades, se utiliza una lista de opciones para elegir la máquina problemática para asegurar que no hay fallos ortográficos. Sin embargo, aunque la encuesta proporcione los datos de forma estructurada, es necesario procesarlos mediante un bloque de código para adaptar la estructura al formato JSON buscado. Esto permite utilizar los nombres de variables deseados, pero, sobre todo, transformar la salida de las *checkboxes* a valores booleanos definidos, en lugar de las listas que genera n8n. El tiempo de procesado es prácticamente nulo.

Si se recibe un formulario digital en formato PDF, se extrae todo su contenido (versión de PDF, idioma, fecha de creación, quién lo creó, número de páginas, etc.) y se filtra para quedarse únicamente con el texto plano (el cual se limpia para eliminar los saltos de línea (“\n”). A

continuación, se utiliza una instancia ejecutada en local de Ollama para pedirle a un LLM (qwen2.5:3b) que extraiga la información de ese texto limpio y genere un JSON a partir del mismo, especificándole la estructura del JSON deseada. Por último, se extrae el JSON del mensaje de respuesta recibido con un bloque de código JavaScript, ya que la respuesta del LLM se recibe como una gran cadena de texto cuyo único contenido es el JSON. Todo este proceso tarda alrededor de 1 minuto cuando se ejecuta sobre un portátil equipado de un procesador Intel i7 de octava generación. Tras varios ejercicios de *prompt engineering*, se decidió utilizar el siguiente *prompt*:

Tabla 7 Prompt para generar un JSON

Prompt para generar un JSON
Eres un extractor de datos. Tu tarea es EXTRAER información del texto y devolver SOLO un JSON válido. REGLAS ESTRUCTAS: - NO inventes información. Si un campo no aparece, deja valor vacío "" o -1 o false según corresponda - NO añadas texto explicativo ni comentarios - NO modifiques los nombres de los campos del JSON de ejemplo - NO modifiques la estructura del JSON de ejemplo Este es el JSON de ejemplo. Cambia únicamente los valores de sus campos: <pre>{{ \$json.ejemplo_json }}</pre> El texto del que debes extraer los datos es el siguiente: <pre>{{ \$json.text }}</pre>
Fuente: Elaboración propia

Para procesar imágenes de formularios manuscritos, se utiliza una API implementada con FastAPI en Python que espera solicitudes que contengan una imagen, la cual se carga en local y se utiliza OCR para extraer el texto que se detecte en la misma. Una vez realizado esto, se reutiliza el *workflow* para generar el JSON a partir de PDFs, es decir, se le da al LLM un JSON de ejemplo y el texto del que debe extraer los datos para construir el JSON final. Esta estrategia permite un gran modularidad, ya que puede cambiarse el algoritmo de OCR que se utiliza desde la API sin necesidad de modificar el *workflow* n8n (siempre mandará la misma *request POST HTTP* a <http://ocr:8000/ocr>).

Otra opción sería utilizar un *Vision LLM* capaz de hacer tanto el OCR como de generar el JSON, como MiniCPM-V, Qwen2.5-VL o Llama 3.2 Vision, pero estos modelos requieren de mucha RAM y son lentos y complejos, además de suponer una pérdida de modularidad.

Para el OCR, se intentó utilizar Tesseract OCR, ya que es muy fácil de instalar y ejecutar en local dada su velocidad en CPU, pero no funcionó bien para extraer el texto de imágenes (es más indicado para leer documentos digitales o escaneados). Por ello, también se implementó otra versión de la API que utiliza PaddleOCR, mucho más potente y casi igual de rápido.

5.2.2. Validación del OCR para ingesta documental

En este apartado se proporcionan ejemplos de los documentos utilizados para validar el OCR de la ingesta documental. Se utilizaron imágenes de formularios totalmente impresos, formularios totalmente manuscritos y formularios híbridos (con partes impresas completadas con escritura a mano). Después de cada imagen de ejemplo se muestran los JSONs resultantes de aplicar OCR con Tesseract y PaddleOCR.

5.2.2.1. Formulario totalmente impreso

Tabla 8 Formulario impreso para validar la ingesta documental con OCR.

Reporte de Incidencia

Fecha (AAAA-MM-DD): 2026-05-18

Máquina problemática: MetroPT3

Nombre del operario: José Miguel Torres Cámara

Descripción del problema:

La máquina está funcionando bajo carga sin problema, pero hace un ruido extraño.

Valores de las variables*:

- Presiones:
 - Del compresor (TP2): 12.4
 - Del panel neumático (TP3): 10.9
 - Al descargar el filtro ciclónico (H1): 3.6
 - Al descargar los secadores (DV): 0
 - Del depósito de aire (R): 11.1
- Señales eléctricas:
 - COMP: 0
 - DV Electric: 1
 - TOWERS: 1
 - MPG: 0
 - LPS: 0
 - Pressure Switch: 0
 - Oil Level: 0
- Corriente del motor: 6.8
- Temperatura del aceite: 63

* Indicar las presiones en bares (bar), la corriente en amperios (A), la temperatura en grados centígrados (°C). Para las señales eléctricas, utilizar "1" si está encendida y "0" si está apagada.

Fuente: Elaboración propia

Tabla 9 JSONs obtenidos a partir de formularios totalmente impresos.

Tesseract	PaddleOCR
<pre>{ "fecha": "2026-05-18", "maquina": "MetroPT3", "operario": "José Miguel Torres Cámara", "descripcion": "A máquina está funcionando bajo carga sin problema, pero hace un ruido extraño.", "variables": { "presiones": { "tp2": 0, "tp3": 0, "h1": 0, "dv": 1, "r": 1 }, "senales_electricas": { "comp": false, "dv_electric": true, "towers": true, "mpg": false, "lps": false, "pressure_switch": false, "oil_level": false }</pre>	<pre>{ "fecha": "", "maquina": "MetroPT3", "operario": "José Miguel Torres Camara", "descripcion": "La máquina está funcionando bajo carga sin problema, pero hace un ruido extrao.", "variables": { "presiones": { "tp2": 12.4, "tp3": 10.9, "h1": 3.6, "dv": 0, "r": 11.1 }, "senales_electricas": { "comp": false, "dv_electric": true, "towers": true, "mpg": false, "lps": false, "pressure_switch": false, "oil_level": false }</pre>

<pre>}, "corriente_motor": 6.8, "temperatura_aceite": 63 } }</pre>	<pre>}, "corriente_motor": 6.8, "temperatura_aceite": 63 } }</pre>
--	--

Fuente: Elaboración propia

PaddleOCR extrae toda la información correctamente, excepto algunos signos de puntuación y la fecha. Tesseract da un rendimiento similar, pero no extrae los valores de las presiones.

5.2.2.2. Formulario híbrido

Tabla 10 Formulario híbrido para validar la ingesta documental con OCR.

Reporte de Incidencia

Fecha (AAAA-MM-DD): 2026-05-18

Máquina problemática: METRO PT3

Nombre del operario: JOSÉ MIGUEL TORRES CÁMARA

Descripción del problema:

LA MÁQUINA ESTÁ FUNCIONANDO, PERO HACE UN RUIDO EXTRAÑO.

Valores de las variables*:

- Presiones:
 - Del compresor (TP2): 12.4
 - Del panel neumático (TP3): 10.9
 - Al descargar el filtro ciclónico (H1): 3.6
 - Al descargar los secadores (DV): 0
 - Del depósito de aire (R): 11.1
- Señales eléctricas:
 - COMP: 0
 - DV electric: 1
 - TOWERS: 1
 - MPG: 0
 - LPS: 0
 - Pressure Switch: 0
 - Oil Level: 0
- Corriente del motor: 6.8
- Temperatura del aceite: 63

* Indicar las presiones en bares (bar), la corriente en amperios (A), la temperatura en grados centígrados (°C). Para las señales eléctricas, utilizar "1" si está encendida y "0" si está apagada.

Tabla 11 JSONs obtenidos a partir de formularios parcialmente manuscritos.

Tesseract	PaddleOCR
<pre>{ "fecha": "", "maquina": "", "operario": "", "descripcion": "", "variables": { "presiones": { "tp2": 0, "h1": 0, "dv": 0, "r": 0 }, "senales_electricas": { "comp": false, "towers": false, "mpg": false, "lps": false, "pressure_switch": false, "oil_level": false }</pre>	<pre>{ "fecha": "2026-05-18", "maquina": "METRo PT 3", "operario": "JostMIGvEl ToRRES cAMARA", "descripcion": "LA MAQVINA ESTA FUNCIONANDO PERO HACE UN RVIDO EXTRANO.", "variables": { "presiones": { "tp2": 12, "tp3": 10.1, "h1": 3.6, "dv": 0, "r": 11.1 }, "senales_electricas": { "comp": 0, "dv_electric": 0, "towers": 1, "mpg": 0, "lps": 0, "pressure_switch": 0, "oil_level": 0 }</pre>

<pre>}, "corriente_motor": 0, "temperatura_aceite": 0 } }</pre>	<pre>}, "corriente_motor": 6.8, "temperatura_aceite": 63 } }</pre>
---	--

Fuente: Elaboración propia

PaddleOCR extrae la mayoría de la información bien, aunque confunde algunos caracteres (2 transformado en Z, u transformada en v, etc.). La descripción del problema sigue comprensible y los valores de las variables son casi totalmente correctos (a excepción de uno de los booleanos y algunos de los decimales de las presiones). Por su parte, Tesseract no es capaz de extraer ninguna información manuscrita. Además, ni siquiera encuentra los campos de la presión *TP3* ni del booleano *dv_electric*.

5.2.2.3. Formulario totalmente manuscrito

Tabla 12 Formulario manuscrito para validar la ingesta documental con OCR.

REPORTE DE INCIDENCIA

FECHA (AAAA-MM-DD): 2026-05-18

MÁQUINA PROBLEMÁTICA: Metro PT3

NOMBRE DEL OPERARIO: JOSÉ MIGUEL TORRES CÁMARA

DESCRIPCIÓN DEL PROBLEMA:
La máquina funciona bien, pero hace un ruido extraño.

VALORES DE LAS VARIABLES:

- PRESIONES:
 - TP2: 12.4
 - TP3: 10.9
 - H1: 3.6
 - DV: 0
 - R: 11.1
- SEÑALES:
 - COMP: 0
 - DV ELECTRIC: 1
 - TOWERS: 1
 - MPG: 0
 - LPS: 0
 - PRESSURE SWITCH: 0
 - OIL LEVEL: 0
- CORRIENTE DEL MOTOR: 6.8
- TEMPERATURA DEL ACEITE: 63

Fuente: Elaboración propia

Tabla 13 JSONs obtenidos a partir de formularios totalmente manuscritos.

Tesseract	PaddleOCR
<pre>{ "fecha": "", "maquina": "", "operario": "", "descripcion": "Y e Y e A A A a 2 MP EAYNYANYNDONOA5biADDODASONODOODOOA DU: VSANNXSNSE ANOGOGO OOO WI API WN NU)", "variables": { "presiones": { "tp2": 0, "tp3": 0, "h1": 0, "dv": 0, "r": 0 }, "senales_electricas": { "comp": false, "dv_electric": false, "towers": false, "mpg": false, "lps": false, "pressure_switch": false,</pre>	<pre>{ "fecha": "", "maquina": "MetroPT 3", "operario": "", "descripcion": "La mo'nina", "variables": { "presiones": { "tp2": 12.4, "tp3": 70.9, "h1": 3.6, "dv": 0, "r": 11.1 }, "senales_electricas": { "comp": false, "dv_electric": true, "towers": true, "mpg": false, "lps": false, "pressure_switch": false,</pre>

<pre>"oil_level": false }, "corriente_motor": 0, "temperatura_aceite": 0 } }</pre>	<pre>"oil_level": false }, "corriente_motor": 6.8, "temperatura_aceite": 63 } }</pre>
Tesseract	PaddleOCR

Fuente: Elaboración propia

Tesseract vuelve a ser totalmente incapaz de extraer ningún tipo de información, lo que era esperable dada la complejidad de un informe totalmente manuscrito. Sin embargo, PaddleOCR, aunque no logra extraer algunos campos como la fecha, el operario o la descripción completa, es capaz de extraer los valores correctos para todas las variables (excepto para TP3, que cambia un “1” por un “7”). El rendimiento es mejorable, pero notablemente superior al de Tesseract.

5.2.2.4. Conclusiones de la validación

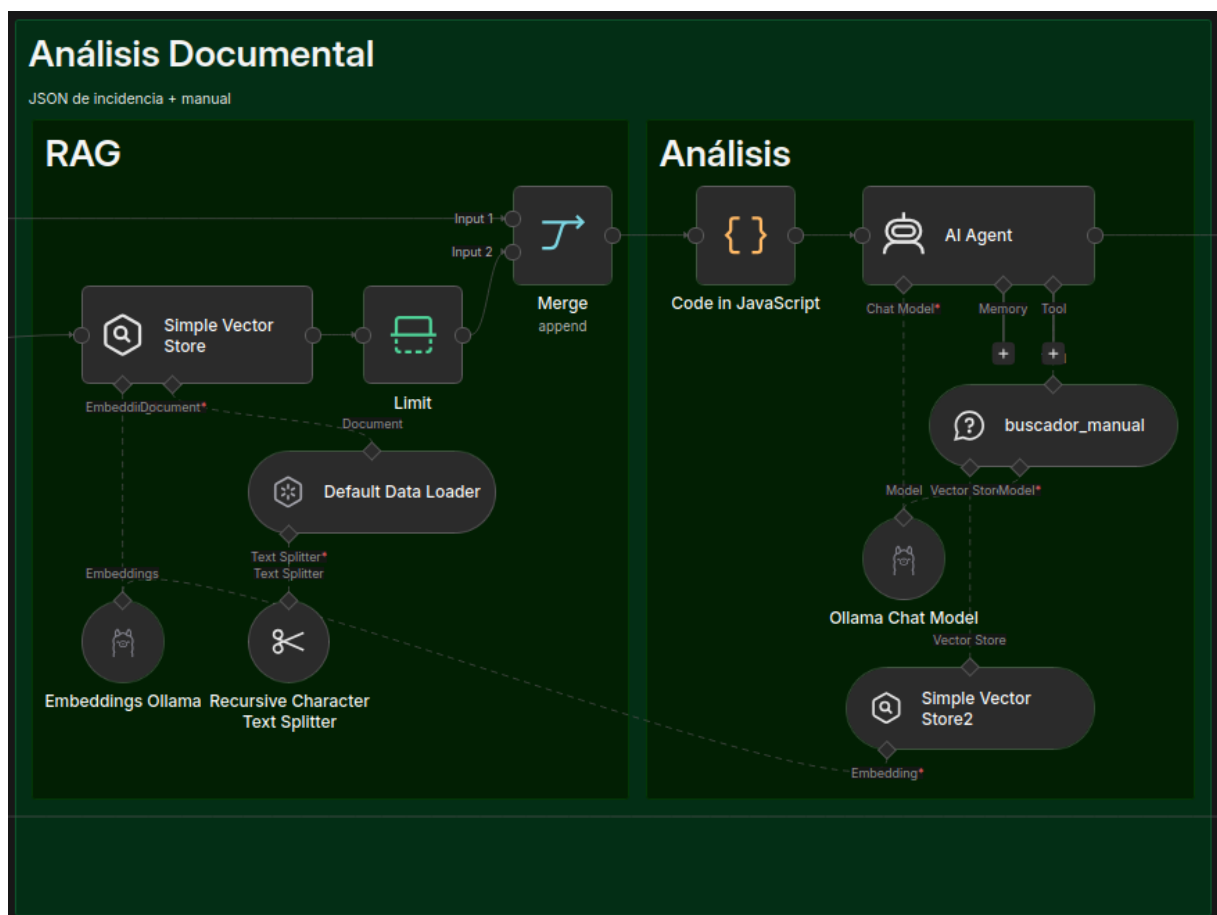
Tras los experimentos realizados, se confirma la sospecha inicial de que Tesseract no es lo suficientemente potente como para extraer texto manuscrito de fotografías. Por su lado, PaddleOCR, a pesar de tener algunos fallos, logra extraer la mayoría de la información correctamente. Aunque desarrollar un OCR con mejor rendimiento sería una mejora sustancial para el sistema, se considera fuera del alcance del proyecto ya que es únicamente una de las tres posibles formas de ingesta, además de ser la menos práctica y cómoda por parte de los usuarios. De todas formas, se ha reflexionado sobre cómo mejorarlo y las opciones de trabajo futuro se exponen en la sección 8.2.

5.2.3. Agente de análisis documental

El Agente de Análisis Documental es el componente de la arquitectura encargado de la ingesta, procesamiento, indexación y posterior consulta de fuentes de información técnica no estructurada (como manuales de maquinaria, normativas de seguridad, PDFs de mantenimiento y guías operativas). Para este caso, el manual (MetroPT-3) del cual leerá la información el sistema está en formato *Markdown*.

Su objetivo principal es dotar al sistema de la capacidad para dar respuesta a las incidencias reportadas de los operarios basándose estrictamente en la documentación oficial de la organización, mitigando las alucinaciones de los modelos de lenguaje (LLMs) mediante una arquitectura de Generación Aumentada por Recuperación (RAG). En la siguiente imagen, se pueden observar los nodos que conforman este agente.

Figura 6 Módulo de análisis documental en n8n

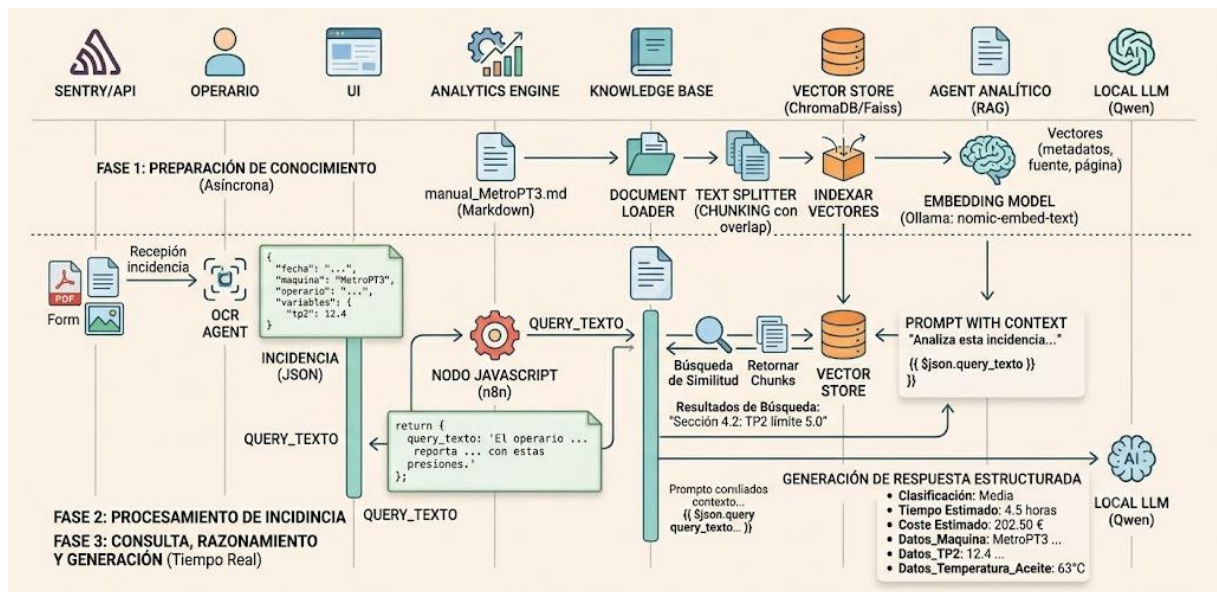


Fuente: Elaboración propia.

5.2.3.1. Workflow y procesamiento de datos (*pipeline RAG*)

El flujo operativo se divide en dos fases críticas: la fase de ingesta (*Offline*), encargada de la preparación de la base de conocimiento, y la fase de consulta y diagnóstico (*Online*), que procesa las incidencias en tiempo real.

Figura 7 Flujo operativo de los datos



Fuente: Elaboración propia.

5.2.3.2. Fase de ingesta y vectorización (*pipeline de datos*)

- **Carga de Documentos (*Document Loading*)**: El agente accede al repositorio de almacenamiento donde se encuentra el documento de referencia de la máquina (`manual_MetroPT3.md`) en formato *Markdown*. El uso de este formato nativo permite preservar estructuras jerárquicas limpias (encabezados #, ##, listas y tablas) sin el ruido o elementos espurios característicos de los archivos PDF estructurados.
- **Fragmentación de Texto (*Chunking*)**: Los modelos de lenguaje tienen limitaciones severas en su ventana de contexto y tienden a diluir la información si se les proveen documentos masivos. Por ello, el manual se divide en fragmentos más pequeños (*chunks*). Esta fragmentación es estrictamente necesaria por tres razones:

1. **Granularidad semántica:** Permite aislar escenarios de fallo, tablas de presión o procedimientos específicos sin mezclarlos con capítulos irrelevantes.
 2. **Eficiencia de cómputo:** Reduce la cantidad de tokens que el LLM local debe procesar en cada inferencia, optimizando los tiempos de respuesta del servidor.
 3. **Precisión en la recuperación:** Garantiza que el algoritmo de búsqueda vectorial apunte directamente al párrafo exacto que resuelve la avería. Se aplica, además, un solapamiento (*overlap*) entre fragmentos contiguos para asegurar que los límites de las frases no corten valores numéricos o condiciones técnicas críticas.
- **Generación de *embeddings*:** Cada fragmento de texto se envía al motor local de Ollama, utilizando el modelo especializado *nomic-embed-text:latest*. Este proceso es indispensable ya que los ordenadores no entienden el lenguaje humano de forma relacional; el modelo de *embeddings* actúa como un traductor matemático que convierte cadenas de texto plano en vectores numéricos de alta dimensionalidad. Estos vectores posicionan geométricamente las ideas en un espacio vectorial, dos textos que hablen de conceptos similares (ej. "*sobrepresión*" y "*exceso de bar*") quedarán ubicados muy cerca el uno del otro, independientemente de las palabras exactas empleadas.
 - **Almacenamiento vectorial (*vector store*):** Los vectores generados se indexan dentro de una Base de Datos Vectorial o Vector Store. La utilidad crítica de este componente reside en su capacidad para indexar y buscar información no por coincidencia exacta de palabras clave (como un buscador tradicional), sino por similitud semántica. Actúa como un índice optimizado de memoria a largo plazo que permite al sistema ejecutar operaciones matemáticas de distancia (como la similitud del coseno) en milisegundos, localizando de forma determinista qué fragmentos del manual técnico corresponden conceptualmente con el problema reportado.

5.2.3.3. Fase de consulta e inferencia en n8n

A diferencia de los sistemas RAG convencionales basados en preguntas directas de usuarios, este agente opera integrándose en un flujo automatizado de incidencias:

- **Entrada del nodo:** El flujo comienza cuando un operario remite un reporte en formato PDF, formulario web o imagen. El agente de extracción previo procesa dicha entrada mediante OCR y genera un archivo JSON estructurado con las variables del entorno.
- **Procesamiento de la incidencia (nodo JavaScript):** El manual y el JSON confluyen en un nodo de tipo *Merge* que unifica el flujo hacia un script de código en JavaScript. Este código extrae de forma segura las métricas físicas y descriptivas de la incidencia en la estructura interna de n8n.
- **Orquestación del *prompt* y ejecución de la IA:** La cadena de texto resultante (`query_texto`) es inyectada directamente en el *prompt* del nodo de la IA mediante la expresión: Analiza esta incidencia y busca la solución en el manual: {{ \$json.query_texto }}

El motor de inferencia se apoya en el modelo local Qwen, el cual está parametrizado para actuar bajo condiciones deterministas (Temperatura = 0.0) y gobernado por un conjunto estricto de instrucciones de ingeniería de *prompts*.

Tabla 14 Configuración de los parámetros de inferencia

Configuración de parámetros
Modelo Base: qwen2.5:3b (local mediante Ollama).
Temperatura: Configurada estrictamente en 0.0 para anular la aleatoriedad y comportamiento creativo del modelo, forzándolo a una evaluación matemática y predictiva.
Herramientas Disponibles: El modelo dispone del acceso exclusivo a la herramienta externa <code>buscador_manual</code> , la cual interactúa directamente con la Vector Store para traer el contexto técnico del archivo <i>Markdown</i> .

Fuente: Elaboración propia

El comportamiento lógico del modelo y las restricciones operativas de la herramienta se definen a través del siguiente mensaje del sistema en el nodo del agente IA.

Tabla 15 Instrucciones técnicas críticas (System Message)

INSTRUCCIÓN TÉCNICA FUNDAMENTAL: Cada vez que utilices la herramienta «buscador_manual», la «entrada» DEBE ser una única cadena de texto sin formato que contenga ÚNICAMENTE los síntomas principales (por ejemplo, «ruido extraño» o «máquina lenta»).

Queda estrictamente PROHIBIDO introducir valores de barras, métricas de sensores, números, temperaturas u objetos JSON en la consulta de la herramienta. Extraiga ÚNICAMENTE las palabras que describen la anomalía física y realice la búsqueda utilizando solo texto.

Eres un experto en mantenimiento industrial especializado en la Unidad de Producción de Aire (APU) MetroPT3. Tu misión es diagnosticar incidencias utilizando exclusivamente la información del manual_MetroPT3.md.

Instrucciones de Razonamiento Obligatorias:

1. Analiza el texto recibido y extrae el síntoma principal (ej: "ruido extraño" o "va lenta").
2. Llama a la herramienta 'buscador_manual' pasando ÚNICAMENTE ese síntoma en texto plano (sin números de presión ni unidades).
3. Busca en la información devuelta por el manual el escenario correspondiente para identificar la Criticidad, el Tiempo Estimado y las Tarifas aplicables.
4. Si la herramienta 'buscador_manual' responde que no encuentra información o no hay datos exactos, NO te disculpes en inglés ni cambies la estructura. Por defecto, determina: Clasificación: Baja, Tiempo Estimado: 2 horas, Coste Estimado: 90 €.

Formato Obligatorio de Salida: Al final de tu respuesta es estrictamente obligatorio incluir este bloque de texto plano. No alteres las etiquetas bajo ningún concepto para evitar errores en la base de datos:

Clasificación: [Alta/Media/Baja]

Tiempo Estimado: [Solo el número] horas

Coste Estimado: [Solo el número calculado] €

Datos_Maquina: MetroPT3

Datos_Operario: [Nombre del operario]

Datos_Descripcion: [Breve resumen del problema en una sola línea]

Datos_TP2: [Solo el número de la presión TP2, o 0 si no hay]

Datos_TP3: [Solo el número de la presión TP3, o 0 si no hay]

Datos_H1: [Solo el número de la presión H1, o 0 si no hay]

Datos_DV: [Solo el número de la presión DV, o 0 si no hay]

Datos_R: [Solo el número de la presión R, o 0 si no hay]

Datos_Comp: [Solo el número, o 0 si no hay]

Datos_DV_Electric: [Solo el número, o 0 si no hay]

Datos_Towers: [Solo el número, o 0 si no hay]

Datos_MPG: [Solo el número, o 0 si no hay]

Datos_LPS: [Solo el número, o 0 si no hay]

Datos_Pressure_Switch: [Solo el número, o 0 si no hay]

Datos_Oil_Level: [Solo el número, o 0 si no hay]

Datos_Corriente_Motor: [Solo el número, o 0 si no hay]

Datos_Temperatura_Aceite: [Solo el número de la temperatura, o 0 si no hay]

Fuente: Elaboración propia

Este blindaje algorítmico asegura que, a pesar de recibir métricas numéricas complejas desde el nodo JavaScript, el LLM realice búsquedas conceptuales (limpias de ruido matemático) en la base de datos vectorial de Ollama, procese la información y devuelva un bloque estructurado con campos fijos que el sistema puede leer directamente para actualizar la base de datos relacional de la plataforma.

5.2.4. Módulo de validación de JSON

El módulo de validación actúa como gate de calidad entre la etapa de ingesta documental y el agente de análisis. Su propósito es garantizar que el JSON generado por el agente extractor sea estructuralmente correcto y semánticamente coherente antes de entregarlo al agente analista: un reporte con datos malformados o incoherentes produciría recomendaciones sin fundamento por parte del modelo de lenguaje.

La lógica de validación reside en una función JavaScript pura (`validarIncidencia`) implementada en `src/validacion/validador.js`, sin dependencias externas, y desarrollada siguiendo la metodología TDD (*Test-Driven Development*): los 19 casos de prueba que especifican su comportamiento se escribieron antes que la implementación y se detallan en la sección de evaluación del sistema. Esta función se inyecta en un nodo Code de n8n mediante un script de *build* (`build-n8n.js`), garantizando que el código verificado en desarrollo y el que se ejecuta en producción sean idénticos y nunca diverjan.

5.2.4.1. Capa 1 — Validación estructural (bloqueante)

La primera capa verifica que el reporte contenga todos los campos necesarios para que el sistema pueda operar sobre él. Si alguna comprobación falla, el campo `valido` del resultado se establece en `false` y el flujo no continúa al agente de análisis. Las comprobaciones realizadas se resumen en la siguiente tabla.

Tabla 16 Detalle de validación estructural

Comprobación	Detalle
Campos de texto obligatorios	<code>fecha</code> , <code>maquina</code> , <code>operario</code> , <code>descripcion</code> presentes y no vacíos
Bloque de variables completo	Presencia de <code>presiones</code> , <code>senales_electricas</code> , <code>corriente_motor</code> , <code>temperatura_aceite</code>
Presiones	<code>tp2</code> , <code>tp3</code> , <code>h1</code> , <code>dv</code> , <code>r</code> — numéricas y ≥ 0

Señales eléctricas	comp, dv_electric, towers, mpg, lps, pressure_switch, oil_level — binarias (0 ó 1)
Corriente del motor	Numérica y ≥ 0
Temperatura del aceite	Numérica

Fuente: Elaboración propia

5.2.4.2. Capa 2 — Validación semántica (informativa, no bloqueante)

La segunda capa detecta inconsistencias de coherencia derivadas del manual técnico de la máquina MetroPT-3. A diferencia de la primera capa, estas inconsistencias no bloquean el flujo: el reporte continúa al agente de análisis, pero las alertas se almacenan en la base de datos y se visualizan en el *dashboard* de monitorización. Cada alerta incluye un código identificador, un mensaje descriptivo y un nivel de severidad. Las reglas implementadas se recogen en la siguiente tabla.

Tabla 17 Detalle de reglas para alertas

Código	Regla	Base en el manual	Severidad
COMP_DV_ELECTRIC	COMP (sin carga) y DV_Electric (bajo carga) no pueden estar activas simultáneamente	Son estados mutuamente excluyentes del compresor	Media
DV_BAJO_CARGA	Si dv_electric=1, la presión dv debe ser ~0 bar	DV debe ser 0 bar cuando el compresor opera bajo carga	Media

R_LEJOS_TP3	La presión del depósito R debe ser cercana a TP3; una diferencia superior a 1,5 bar indica posible fuga o lectura errónea	R debe aproximarse a TP3 en operación normal	Alta
LPS_INCOHERENTE	El sensor LPS se activa si y solo si la presión R es inferior a 7 bar	Umbral de activación del Low Pressure Switch según el manual	Media
CORRIENTE_FUERA_RANGO	La corriente del motor debe estar cercana a los valores nominales {0, 4, 7, 9} A	Valores normales de operación según el manual	Baja

Fuente: Elaboración propia

Los umbrales de estas reglas están centralizados en el objeto UMBRALES dentro de `validador.js`, lo que permite ajustarlos sin modificar la lógica de las reglas. Las inconsistencias estructurales y semánticas no se mezclan: si un reporte falla la capa estructural, la capa semántica no se evalúa, puesto que carecería de sentido razonar sobre coherencia de valores que ni siquiera tienen el tipo correcto.

5.2.4.3. Integración en el *workflow*

En el *workflow* `OpsInsight.json`, los cuatro nodos de validación (`Validar incidencia`, `¿Reporte válido?`, `Reporte inválido` y `Registrar error validacion`) están ubicados en la zona «Validación del JSON», entre la ingesta y el nodo Merge:

```
... → Validar incidencia → ¿Reporte válido? - true → Merge → Análisis → MySQL  
    ↳ false → Reporte inválido → Registrar error validacion → MySQL (validacion_ok = 0)
```

Figura 8 Módulo de validación del JSON en n8n



Fuente: Elaboración propia.

Esta posición garantiza que solo los reportes válidos consumen recursos del agente de análisis (LLM + recuperación semántica). Los reportes inválidos se registran igualmente en la base de datos con `validacion_ok = 0` y los errores serializados en `alertas_validacion`, preservando la trazabilidad completa del proceso de ingesta: un reporte rechazado nunca se descarta silenciosamente. Los reportes válidos que presenten alertas semánticas se registran con `validacion_ok = 1` y las inconsistencias en `alertas_validacion`, para su análisis posterior en el *dashboard*.

5.2.4.4. Persistencia y trazabilidad

Para soportar la monitorización de la calidad de ingesta, la tabla `registro_incidentes` (definida en `db/schema.sql`) incorpora dos columnas específicas:

Tabla 18 Campos para registrar incidencias

Columna	Tipo	Descripción
validacion_ok	TINYINT(1)	1 = válido estructuralmente; 0 = inválido o error de sistema
alertas_validacion	TEXT	JSON con los errores estructurales o las inconsistencias semánticas detectadas

Fuente: Elaboración propia

5.3.REGISTRO DE INCIDENCIAS

El Nodo de Registro constituye la capa de persistencia final y auditoría operativa dentro del *workflow* de OpsInsight Analytics. Su propósito fundamental es doble, por un lado, estructurar y almacenar de forma redundante los diagnósticos exitosos generados por el Agente de Análisis Documental y, por otro, centralizar en la misma infraestructura relacional los reportes que hayan fallado en las etapas previas debido a errores de validación, garantizando así la trazabilidad absoluta de todo documento que ingrese al sistema.

Figura 9 Módulo de registro de incidencias en n8n



Fuente: Elaboración propia.

Como se puede apreciar en la imagen del flujo de datos se bifurca según el estado y el nodo de origen de la información.

Cuando el Agente de Análisis Documental procesa correctamente la incidencia, extrae el bloque rígido de variables y emite el diagnóstico, el flujo se divide de forma simultánea en dos rutas:

- Ruta de Histórico Plano (CSV): Toda la información estructurada por la IA se concatena y escribe secuencialmente en un archivo en formato CSV. Este archivo plano actúa como un registro histórico inmutable y descentralizado, ideal para auditorías externas, exportaciones masivas a herramientas analíticas externas o planes de recuperación ante desastres físicos en la base de datos.
- Ruta de Producción Relacional (MySQL): De manera paralela, las métricas, clasificaciones de criticidad y variables del JSON procesado por la IA se insertan mediante sentencias SQL optimizadas en las tablas operativas de la base de datos relacional MySQL. Esta base de datos da soporte en tiempo real a los *dashboards* e interfaces web de los supervisores de planta.

Si un reporte analógico o digital es rechazado en etapas tempranas del pipeline (por ejemplo, fallos de lectura en el motor OCR, campos obligatorios vacíos o formatos corruptos), el nodo inferior *Registrar Error de Validación* captura de forma automática el error.

- Persistencia Integrada: En lugar de descartar la interacción, el sistema redirige el *payload* fallido directamente a la misma base de datos de MySQL. Esto permite registrar la anomalía bajo un estado específico (ej. Estado: FALLIDO_VALIDACION), indexando el tipo de error técnico para que el equipo de soporte pueda identificar cuellos de botella en la ingesta o formularios mal cumplimentados por los operarios.

El orquestador de agentes se encarga de interceptar y mapear los datos según el camino correspondiente, normalizando la información bajo los siguientes bloques:

1. Datos Operativos e Identificación

- **Metadatos del reporte:** Nombre del operario, máquina afectada (MetroPT3) y un breve resumen textual del problema detectado en una sola línea.

2. Telemetría Completa de Variables (Solo Camino Exitoso)

El sistema indexa las variables físicas continuas y las señales lógicas enviadas originalmente en el JSON y ratificadas por la IA:

- **Presiones:** Valores numéricos en bar para los puntos de control tp2, tp3, h1, dv y r.
- **Componentes eléctricos y niveles:** Estados booleanos o métricas del compresor (Comp), válvulas (DV_Electric), torres de secado (Towers), presostatos (Pressure_Switch) y nivel de aceite (Oil_Level).
- **Variables físicas:** Corriente del motor y temperatura del aceite.

3. Diagnóstico y Parámetros de Gestión

- **Severidad y estimaciones:** Clasificación del fallo (Alta/Media/Baja), el tiempo estimado de intervención técnica medido en horas y el coste monetario de reparación calculado en euros (€).

4. Estampado de Tiempo Estándar (ISO 8601)

Tanto para los registros insertados con éxito como para los errores de validación grabados en MySQL, el sistema genera de forma automática una marca de tiempo en el instante exacto de la inserción. Esta marca sigue estrictamente el estándar internacional ISO 8601 en formato extendido UTC con precisión de milisegundos:

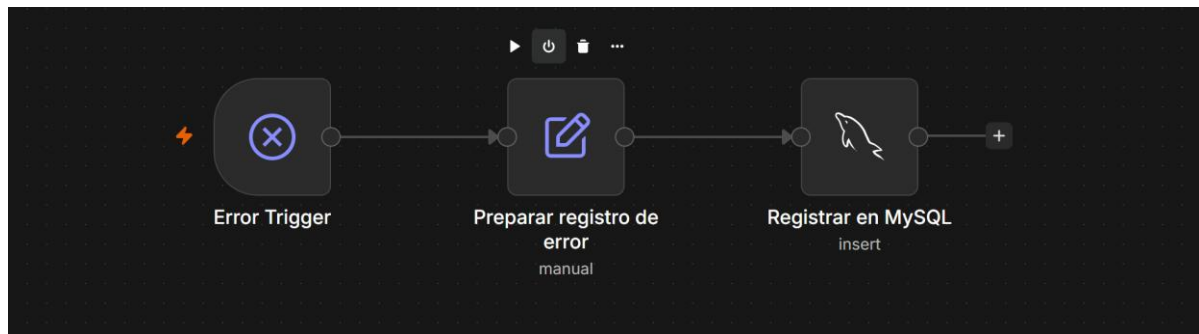
5.4.CONTROL DE ERRORES Y TRAZABILIDAD DEL SISTEMA

El sistema implementa el control de errores en dos niveles complementarios. El primer nivel son los **errores de datos**: reportes malformados o incoherentes, gestionados por el módulo de validación descrito anteriormente, que los intercepta antes del análisis y los registra con `validacion_ok = 0`. El segundo nivel son los **errores de ejecución**: fallos de infraestructura que la validación de datos no puede anticipar, como un *timeout* del modelo de lenguaje en Ollama, una pérdida de conexión con la base de datos, la caída del servicio OCR o una excepción no controlada en cualquier nodo del *workflow*.

Para este segundo nivel se desarrolló un *workflow* dedicado de manejo de errores, *OpsInsight_ErrorHandler* (en *src/OpsInsight_ErrorHandler.json*), enlazado al *workflow* principal mediante el ajuste *errorWorkflow* de *n8n*. Cuando cualquier nodo del flujo principal lanza una excepción no controlada, *n8n* invoca automáticamente este *workflow*, que ejecuta la cadena:

Error Trigger → Preparar registro de error → Registrar en MySQL

Figura 10 Workflow de manejo de errores *OpsInsight_ErrorHandler* en *n8n*



Fuente: Elaboración propia.

El nodo *Error Trigger* recibe de *n8n* el contexto completo del fallo (*workflow* afectado, último nodo ejecutado, mensaje y traza del error). El nodo *Preparar registro de error* lo transforma en un registro compatible con la tabla *registro_incidencias*, de modo que los errores de sistema conviven con las incidencias operativas y son consultables con las mismas herramientas. Los campos registrados son:

Tabla 19 Tabla para registro de incidencias

Campo	Valor registrado
fecha	Marca temporal del fallo en formato ISO 8601 (<code>\$now.toISO()</code>)
maquina	Nombre del <i>workflow</i> afectado

operario	SISTEMA (identifica el registro como automático, no humano)
descripcion	Último nodo ejecutado antes del fallo (Error en nodo: ...)
solucion_propuesta	Mensaje de la excepción capturada
criticidad	SISTEMA_ERROR (permite filtrar errores de sistema en el dashboard)
validacion_ok	0
alertas_validacion	JSON con nodo afectado, mensaje de error y primera línea de la traza

Fuente: Elaboración propia

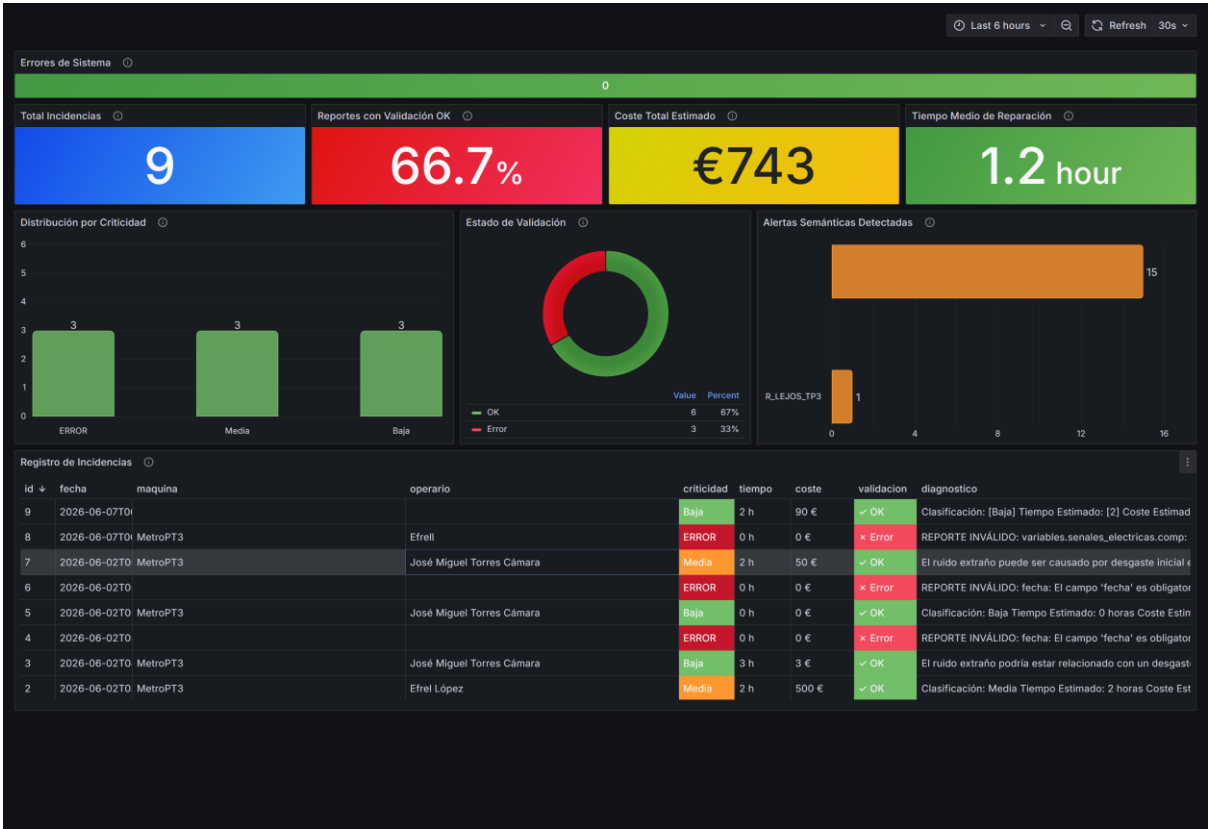
Este diseño aporta tres garantías. Primera, **ningún fallo es silencioso**: todo error de ejecución queda persistido con su contexto. Segunda, **trazabilidad**: el registro identifica el nodo exacto donde se produjo el fallo y conserva la traza para el diagnóstico. Tercera, **observabilidad**: el panel «Errores de Sistema» del *dashboard* de monitorización contabiliza los registros con criticidad SISTEMA_ERROR, de forma que una degradación de la infraestructura es visible de inmediato sin necesidad de revisar los logs de n8n.

5.5.DASHBOARD DE MONITORIZACIÓN

El *dashboard* de monitorización está implementado con Grafana 11.4 y se auto-provisiona al arrancar el stack con `docker compose up -d grafana`, sin necesidad de configuración manual. La fuente de datos es la base de datos MySQL del sistema, conectada mediante el fichero `grafana/provisioning/datasources/opsinsight_mysql.yaml`. El panel de control (`grafana/dashboards/opsinsight.json`) se carga automáticamente mediante el proveedor declarado en `grafana/provisioning/dashboards/provider.yaml`. De este modo, la definición completa del *dashboard* queda versionada como código junto al resto del proyecto: cualquier

miembro del equipo que levante el entorno obtiene exactamente el mismo *dashboard*, y sus cambios se revisan por *pull request* como los del resto del sistema.

Figura 11 Dashboard de monitorización en Grafana con datos sintéticos de validación



Fuente: Elaboración propia.

El *dashboard* incluye los siguientes paneles:

Tabla 20 Paneles configurados en Grafana

Panel	Tipo	Métrica visualizada
Errores de Sistema	Estadístico	Registros con criticidad SISTEMA_ERROR (activaciones del manejador de errores)
Total Incidencias	Estadístico	Total de registros en registro_incidentes

Reportes con Validación OK	Estadístico	Porcentaje de registros con <code>validacion_ok = 1</code>
Coste Total Estimado	Estadístico	Suma del campo <code>coste_estimado</code>
Tiempo Medio de Reparación (MTTR)	Estadístico	Media del campo <code>tiempo_reparacion</code>
Distribución por Criticidad	Gráfico de barras	Recuento de registros agrupados por criticidad
Estado de Validación	Gráfico de donut	Proporción de registros válidos frente a inválidos
Alertas Semánticas Detectadas	Gráfico de barras	Frecuencia de cada código de alerta semántica
Registro de Incidencias	Tabla	Vista completa, con la columna de validación coloreada en verde (OK) o rojo (Error)

Fuente: Elaboración propia

Los paneles «Estado de Validación» y «Alertas Semánticas Detectadas» permiten evaluar de manera continua la calidad del proceso de ingesta: si la tasa de validación cae o aumenta la frecuencia de una alerta semántica concreta, el equipo de operaciones puede identificar el origen del problema directamente desde el *dashboard*, sin necesidad de consultar los registros internos del *workflow*. Junto con el panel «Errores de Sistema», el *dashboard* cubre así las tres dimensiones de la salud del sistema: calidad del dato, coherencia operativa e infraestructura.

5.5.1. Análisis e interpretación de los indicadores operativos

Más allá de su función de visualización, el *dashboard* se concibe como una herramienta de lectura operativa: cada indicador está pensado para que un responsable de mantenimiento, sin formación estadística avanzada, extraiga una conclusión accionable de un vistazo. A continuación, se interpretan los valores observados sobre el conjunto de ejecuciones de prueba acumuladas en la base de datos, que constituye la evidencia del funcionamiento de extremo a extremo del sistema.

Estado de validación. El indicador «Reportes con Validación OK» se situó en el 66,7 % sobre el total de incidencias registradas. Lejos de ser una deficiencia, esta cifra es coherente con el diseño del banco de pruebas, en el que aproximadamente un tercio de los reportes se generó deliberadamente con errores para ejercitar la rama de rechazo. El valor del indicador no reside en su magnitud absoluta sino en su uso como señal de tendencia: en explotación real, una caída sostenida de este porcentaje delataría un problema sistemático de ingesta —un formulario mal diseñado, un operario que cumplimenta mal un campo o una degradación del OCR— antes de que ese ruido contamine el análisis posterior.

Distribución por criticidad. El panel correspondiente desglosa las incidencias según la clasificación —Alta, Media, Baja— que asigna el agente de diagnóstico, además de la categoría `SISTEMA_ERROR` reservada a los fallos de infraestructura. Esta distribución es la base de la priorización operativa: permite identificar de inmediato si la carga de trabajo se concentra en incidencias graves —que exigen intervención inmediata— o en una mayoría de fallos menores, orientando la asignación de recursos del equipo técnico. La convivencia de las incidencias operativas y los errores de sistema en el mismo eje permite, además, distinguir de un vistazo los problemas del activo de los problemas de la propia plataforma.

Coste y tiempo de reparación. Los indicadores «Coste Total Estimado» (743 € acumulados en las pruebas) y «Tiempo Medio de Reparación» (1,2 horas) traducen el historial de incidencias a las dos magnitudes que gobiernan las decisiones de mantenimiento: el dinero y la disponibilidad del activo. Su valor analítico no está en la cifra puntual, sino en su evolución acumulada: son los indicadores que, cruzados con la frecuencia de fallo por activo, permitirían construir el índice de criticidad compuesto previsto en el objetivo OE7. El sistema ya captura y expone las tres magnitudes necesarias —frecuencia, tiempo y coste—; su combinación en

una métrica única de priorización queda planteada como evolución natural una vez disponible un volumen histórico suficiente.

Alertas semánticas. El panel «Alertas Semánticas Detectadas» cuantifica la frecuencia de cada inconsistencia de coherencia. En las pruebas, la alerta predominante fue R_LEJOS_TP3, que señala una divergencia anómala entre la presión del depósito y la de referencia. La utilidad de este panel es doble: a corto plazo, alerta de un posible problema físico recurrente —una fuga, una lectura de sensor errónea—; a medio plazo, la prevalencia de un código concreto orienta sobre qué modos de fallo son más habituales en el activo, una información que alimenta tanto el mantenimiento preventivo como la revisión de los propios procedimientos de inspección.

En conjunto, la lectura de estos indicadores confirma que el *dashboard* cierra la cadena de valor del dato planteada como objetivo general: la información que entra como un reporte heterogéneo —un PDF, una fotografía, un formulario— termina condensada en métricas operativas sobre las que es posible decidir. Conviene subrayar, en coherencia con el alcance del trabajo, que el análisis aquí presentado es de naturaleza descriptiva (OE6): el estudio de series temporales, estacionalidad y correlaciones entre variables, así como el cálculo formal del índice de criticidad (OE7), requieren un histórico de incidencias mayor que el generado en la fase de validación y se detallan como líneas de trabajo futuro.

5.6. EVALUACIÓN DEL SISTEMA: VALIDACIÓN Y MONITORIZACIÓN

5.6.1. Evaluación unitaria del módulo de validación

El módulo de validación se desarrolló siguiendo TDD, con una suite de 19 casos de prueba implementada con el módulo nativo `node:test` (sin dependencias externas) en `src/validacion/validador.test.js`. Cada caso parte de un reporte base válido y coherente —equivalente al JSON de ejemplo del agente de extracción— y muta únicamente el aspecto que quiere probar, lo que aísla cada regla y documenta el comportamiento esperado del módulo. Las tablas siguientes recogen los 19 casos y su resultado.

Casos de la capa estructural (11):

Tabla 21 Casos de pruebas

#	Caso de prueba	Entrada (mutación sobre el reporte base)	Resultado esperado	Resultado
1	Reporte completo y coherente	Sin mutación	valido = true, sin errores	✓
2	Falta campo de nivel superior	Se elimina maquina	valido = false, error en maquina	✓
3	Falta un bloque completo	Se elimina variables.presiones	valido = false, error en variables.presiones	✓
4	Falta una presión concreta	Se elimina tp2	valido = false, error en variables.presiones.tp2	✓
5	Falta una señal eléctrica	Se elimina lps	valido = false, error en variables.senales_electricas.lps	✓
6	Presión no numérica	tp3 = "diez"	valido = false, error en tp3	✓
7	Señal eléctrica no binaria	comp = 2	valido = false, error en comp	✓
8	Presión negativa (físicamente imposible)	r = -3,2	valido = false, error en r	✓
9	Corriente del motor negativa	corriente_motor = -1	valido = false, error en corriente_motor	✓
10	Entrada nula o no objeto	null	valido = false sin lanzar excepción	✓
11	Error estructural no evalúa semántica	Se elimina variables.presiones	inconsistencias = [] (no se mezclan capas)	✓

Fuente: Elaboración propia

Casos de la capa semántica (8):

Tabla 22 Casos capa semántica

#	Caso de prueba	Entrada (mutación sobre el reporte base)	Resultado esperado	Resultado
12	Reporte coherente	Sin mutación	Sin inconsistencias	✓
13	Estados del compresor excluyentes	comp = 1 y dv_electric = 1	Alerta COMP_DV_ELECTRIC (el reporte sigue siendo válido)	✓
14	Presión DV alta bajo carga	dv_electric = 1, dv = 4,5 bar	Alerta DV_BAJO_CARGA	✓
15	Depósito alejado de TP3	tp3 = 10,9, r = 4,0 bar	Alerta R_LEJOS_TP3	✓
16	LPS encendida con presión alta	lps = 1, r = 11,1 bar	Alerta LPS_INCOHERENTE	✓
17	LPS apagada con presión baja	lps = 0, r = 5,0 bar	Alerta LPS_INCOHERENTE	✓
18	Corriente fuera de valores nominales	corriente_motor = 15 A	Alerta CORRIENTE_FUERA_RANGO	✓
19	Estructura completa de la alerta	comp = 1 y dv_electric = 1	La alerta incluye código, mensaje y severidad ∈ {alta, media, baja}	✓

Fuente: Elaboración propia

La suite completa se ejecuta con node --test desde src/validacion/ con resultado de **19/19 casos superados**, y se vuelve a ejecutar tras cualquier cambio en las reglas antes de regenerar el nodo de n8n.

Figura 12 Resultado de la suite de pruebas (19/19 superados)

```
Terminal - node --test
PS C:\Users\efli9\projects\unir_q2a3_tfm\src\validacion> node --test
✓ estructural: el reporte base es válido y sin errores (2.1686ms)
✓ estructural: falta un campo de nivel superior (maquina) (0.2551ms)
✓ estructural: falta el bloque variables.presiones (0.1972ms)
✓ estructural: falta una presión concreta (tp2) (0.2383ms)
✓ estructural: falta una señal eléctrica concreta (lps) (0.2101ms)
✓ estructural: una presión no es numérica (string) (0.19ms)
✓ estructural: una señal eléctrica no es binaria (valor 2) (0.17ms)
✓ estructural: una presión es negativa (físicamente imposible) (0.1757ms)
✓ estructural: la corriente del motor es negativa (0.187ms)
✓ estructural: entrada nula o no objeto produce error sin lanzar excepción (0.2302ms)
✓ estructural: un JSON con error NO produce inconsistencias semánticas (no se evalúan) (0.1847ms)
✓ semántica: reporte base coherente no genera inconsistencias (0.1364ms)
✓ semántica: COMP y DV_Electric ambas activas se contradicen (0.1516ms)
✓ semántica: bajo carga (dv_electric=1) la presión DV debe ser ~0 (0.1601ms)
✓ semántica: R debe ser cercana a TP3 (0.2156ms)
✓ semántica: LPS encendida exige presión < 7 bar (1.1265ms)
✓ semántica: presión < 7 bar exige LPS encendida (0.1613ms)
✓ semántica: corriente del motor muy alejada de los valores nominales (0.1386ms)
✓ semántica: cada inconsistencia incluye código, mensaje y severidad (0.1286ms)
i tests 19
i suites 0
i pass 19
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 134.3263
```

Fuente: Elaboración propia.

5.6.2. Verificación de extremo a extremo

Además de las pruebas unitarias, el flujo completo se verificó en el entorno real (n8n + Ollama + MySQL + Grafana desplegados con Docker Compose) con los siguientes escenarios:

Tabla 23 Escenarios de verificación de extremo a extremo

Escenario	Comportamiento verificado
Reporte válido por formulario online	Supera la validación, es analizado por el agente y se registra con <code>validacion_ok = 1</code>
Reporte válido por PDF	Mismo recorrido que el anterior por la vía de ingesta documental

Reporte válido por imagen (OCR)	Mismo recorrido por la vía de OCR con PaddleOCR
Reporte inválido	La validación lo bloquea antes del análisis y queda registrado con <code>validacion_ok = 0</code> y los errores en <code>alertas_validacion</code>
Reporte válido con incoherencia semántica	Pasa al análisis y la alerta (p. ej. <code>R_LEJOS_TP3</code>) queda registrada y visible en el panel «Alertas Semánticas Detectadas»
Error de infraestructura	El workflow de manejo de errores registra el fallo con criticidad <code>SISTEMA_ERROR</code> y el panel «Errores de Sistema» lo refleja

Fuente: Elaboración propia

Cabe destacar que esta verificación de extremo a extremo permitió detectar y corregir un defecto real de integración: el formulario online generaba las señales eléctricas como valores booleanos (`true/false`), mientras que el esquema de datos exige binario numérico (0/1), por lo que todo reporte enviado por formulario fallaba la validación estructural. El nodo de transformación del formulario se corrigió para emitir 0/1, evidenciando el valor del módulo de validación también como herramienta de control de calidad durante el propio desarrollo del sistema.

El resultado de estas ejecuciones es observable en el *dashboard* de monitorización (véase la figura del apartado anterior): en el momento de la captura, el sistema acumulaba incidencias procedentes de las tres vías de ingesta, con la tasa de validación reflejada en el panel «Reportes con Validación OK», los reportes inválidos marcados en rojo en la tabla de registro y la alerta semántica `R_LEJOS_TP3` contabilizada en el panel de alertas, lo que confirma el recorrido completo del dato desde la ingesta hasta la visualización.

5.6.2.1. Ejemplos de validación de extremo a extremo

Para ilustrar el comportamiento del módulo de validación se presentan tres casos representativos, correspondientes a las tres salidas posibles del sistema: reporte válido y coherente, reporte estructuralmente inválido y reporte válido con inconsistencia semántica.

Caso 1 — Reporte válido y coherente. El JSON supera ambas capas. El campo `_validacion` que el nodo añade al ítem es:

```
{
  "valido":true,
  "errores":[],
  "inconsistencias":[]
}
```

El ítem continúa hacia el Merge y el agente de análisis, y se registra con `validacion_ok = 1` y `alertas_validacion = []`.

Caso 2 — Reporte estructuralmente inválido. Un reporte en el que la señal `comp` llega como 2 (no binaria) y falta el campo `maquina` produce:

```
{
  "valido":false,
  "errores":[
    { "campo": "maquina", "mensaje": "Campo obligatorio ausente o vacío" },
    { "campo": "variables.senales_electricas.comp",
      "mensaje": "La señal 'comp' debe ser binaria (0 o 1)" }
  ],
  "inconsistencias": []
}
```

El nodo ¿Reporte válido? desvía el ítem a la rama `false`: no se invoca al agente y el reporte se registra con `validacion_ok = 0` y los errores serializados en `alertas_validacion`, quedando visible en rojo en la tabla del *dashboard*.

Caso 3 — Reporte válido con inconsistencia semántica. Un reporte estructuralmente correcto pero cuya presión de depósito `r` (4,0 bar) diverge en exceso de `tp3` (10,9 bar) supera la capa estructural pero activa una alerta:

```
{
  "valido": true,
  "errores": [],
  "inconsistencias": [
    { "codigo": "R_LEJOS_TP3",
      "mensaje": "La presión R (4.0) difiere de TP3 (10.9) más de lo admisible",
```

```
"severidad": "alta" }  
]  
}
```

El reporte continúa al análisis (la incoherencia no es bloqueante) y se registra con `validacion_ok = 1`, pero la inconsistencia queda en `alertas_validacion` y se contabiliza en el panel «Alertas Semánticas Detectadas», alertando al operario de una posible fuga o lectura errónea sin detener la operación.

5.7.DISCUSIÓN DE LOS RESULTADOS

Los resultados de la verificación de extremo a extremo confirman que las tres vías de ingesta convergen correctamente en el esquema JSON común y que el módulo de validación discrimina de forma fiable entre reportes utilizables e inválidos. Sobre el conjunto de ejecuciones de prueba acumuladas en el *dashboard*, la tasa de reportes que superan la validación estructural se sitúa en el entorno del 65–70 %, una cifra coherente con el diseño del experimento: una parte de los reportes se construyó deliberadamente con errores (campos ausentes, señales no binarias, presiones negativas) para ejercitar la rama de rechazo y comprobar que esos casos quedan registrados con `validacion_ok = 0` en lugar de descartarse silenciosamente.

La capa semántica demostró su utilidad al detectar inconsistencias que la validación estructural no puede capturar. El caso más frecuente en las pruebas fue la alerta `R_LEJOS_TP3`, que señala una divergencia anómala entre la presión del depósito y la de referencia TP3 —un patrón asociado en el manual a posibles fugas o lecturas erróneas—. Estas alertas no bloquean el análisis pero quedan visibles en el panel correspondiente, ofreciendo al operario una señal temprana de calidad del dato sin frenar la operación.

Un hallazgo relevante del proceso de integración fue la detección de un defecto real gracias al propio módulo de validación: el formulario online emitía las señales eléctricas como valores booleanos (`true/false`) cuando el esquema exige binario numérico (0/1), por lo que todo reporte enviado por formulario fallaba la validación. La corrección de un único nodo de transformación resolvió el problema. Este episodio ilustra el valor de la validación determinista no solo en producción, sino como herramienta de control de calidad durante el propio desarrollo del sistema.

En cuanto al rendimiento, el cuello de botella del pipeline reside en la inferencia del modelo de lenguaje local: la extracción del JSON a partir de un PDF y el posterior diagnóstico con RAG tardan, en conjunto, en torno a un minuto sobre una CPU de gama media, frente a los tiempos prácticamente nulos de la validación y la inserción en base de datos. Este resultado confirma la decisión de mantener la inferencia fuera de la ruta crítica de validación —rechazando los reportes inválidos antes de invocar al agente— y orienta las líneas de mejora hacia la aceleración por GPU descritas en el capítulo de trabajo futuro.

6. Código fuente y datos analizados

6.1. Código fuente

Para desarrollar el sistema documentado en esta memoria, se utiliza GitHub como herramienta de control de versiones y colaboración entre compañeros. Todo el código se aloja en el siguiente repositorio: github.com/jmtc7/unir_q2a3_tfm. Dicho repositorio contiene un fichero README.md que documenta su estructura y explica cómo instalar y ejecutar el sistema desarrollado. Específicamente, se utiliza Docker Compose para crear múltiples contenedores Docker que interactúan entre ellos para ejecutar todas las tareas necesarias. Estas tareas o componentes son una API para el OCR, Ollama para ejecutar LLMs en local, Grafana para monitorizar el sistema, MySQL para gestionar la base de datos y n8n para orquestar y automatizar todos los procesos (y los errores que surjan durante la ejecución).

6.1.1. Reproducibilidad y puesta en marcha

El repositorio está concebido para que un tercero reproduzca el sistema completo sin conocimiento previo de su arquitectura interna. Una vez clonado el repositorio e instalado Docker, la puesta en marcha consta de tres pasos. Primero, se levantan los servicios con `docker compose up -d`. Segundo, se descargan en Ollama los dos modelos que utiliza el sistema: `qwen2.5:3b` para la extracción de JSON y el diagnóstico, y `nomic-embed-text` para la generación de los *embeddings* del manual técnico. Tercero, se importa el *workflow* `OpsInsight.json` en n8n y se configuran las credenciales de MySQL, que n8n no exporta por seguridad.

La elección del modelo `qwen2.5:3b` (3.000 millones de parámetros) responde a un compromiso entre capacidad y consumo: requiere apenas unos 2 GB de memoria y se ejecuta sobre CPU, lo que mantiene el sistema accesible en hardware modesto, a costa de un tiempo de inferencia superior al de modelos en la nube. La modularidad del diseño permite sustituir este modelo por uno mayor —o por una API externa— modificando únicamente la credencial de Ollama, sin tocar la lógica del *workflow*. Del mismo modo, el algoritmo de OCR puede cambiarse desde la API del servicio `ocr` sin alterar el *workflow*, que siempre realiza la misma petición HTTP, garantizando la extensibilidad del sistema.

6.2. Datos analizados

Los datos empleados en la validación de OpsInsight Analytics proceden de documentación técnica sintética diseñada tanto para cumplir los requisitos como para desafiar las capacidades de los módulos de análisis e ingesta de PDFs y de imágenes con OCR. Esta documentación se creó tras analizar varios conjuntos de datos públicos de referencia sugeridos por Sopra Steria.

Como referencias de datos tabulares de mantenimiento y fallos se utilizan principalmente el AI4I 2020 Predictive Maintenance Dataset (Matzka, 2020), disponible bajo licencia CC BY 4.0, y el MetroPT-3 Dataset (Veloso et al., 2022), igualmente con licencia CC BY 4.0. El primero ofrece registros sintéticos de sensores industriales con etiquetas de tipo de fallo, adecuado para validar el análisis de criticidad y frecuencia de fallos por activo. El segundo proporciona series temporales de sensores del sistema de compresor de aire del metro de Oporto, especialmente útil para validar la detección de patrones de degradación y comportamiento previo al fallo. Como fuente complementaria orientada a pronóstico, se puede recurrir al *Prognostics Data Repository de la NASA* (NASA, 2007), que incluye datos de rodamientos y turbinas con ciclos de degradación documentados.

A partir de la arquitectura del compresor descrita en MetroPT-3, se han seleccionado variables críticas que actúan como indicadores de salud (*proxies*) del sistema para fundamentar el diagnóstico del agente IA:

- TP2 (Presión del Compresor): Es la variable de salida directa; una caída anómala es el síntoma principal de pérdida de eficiencia volumétrica.
- R (Presión del Depósito) como Proxy de Fugas: Esta variable es fundamental al contrastarse con la presión de referencia TP3. Una divergencia superior a 1,5 bar entre ambas (alerta R_LEJOS_TP3) actúa como un indicador indirecto de la integridad del sellado neumático o de errores de lectura en los sensores.
- Corriente del Motor como Proxy de Carga: La corriente eléctrica consumida es un proxy del esfuerzo mecánico. Valores alejados de los rangos nominales (4A en vacío, 7A en carga) permiten al sistema validar si el estado lógico reportado por el operario es coherente con el consumo energético real del activo.
- Temperatura del Aceite: Funciona como un indicador térmico de la fricción interna y la degradación de la lubricación, siendo un precursor crítico de desgaste prematuro

Es fundamental reconocer las limitaciones inherentes a la metodología y a las fuentes utilizadas para no presentar el modelo como una solución infalible:

1. Naturaleza *snapshot* del Reporte: A diferencia de los sensores IoT que capturan series temporales de alta frecuencia, la información tratada proviene de reportes manuales. Esto limita el análisis a fotografías puntuales del estado de la máquina, lo que impide detectar tendencias de degradación muy lenta o transitorios rápidos que solo se captan en tiempo real.
2. Sesgo de Muestreo y Datos Sintéticos: Al utilizar documentación sintética para validar la robustez del OCR y la validación semántica, el *dataset* puede presentar una tasa de anomalías alta. En un entorno operativo real con mantenimiento preventivo, la frecuencia de estos fallos sería menor, lo que podría generar un sesgo de "falsos positivos" en un modelo no ajustado.
3. Dependencia de la Calidad de la Ingesta: El análisis está condicionado por la precisión del PaddleOCR. Si el operario tiene una caligrafía deficiente o la fotografía es de baja calidad, existe el riesgo de procesar valores erróneos (como confundir un "1" con un "7") que, a pesar de ser validados estructuralmente, podrían derivar en un diagnóstico incorrecto.

Para el módulo documental —ingesta de manuales técnicos y extracción de procedimientos de mantenimiento— se analizaron manuales de acceso público obtenidos de ManualsLib (<https://www.manualslib.com/>), Internet Archive (<https://archive.org/details/manuals>) y los portales de soporte de fabricantes industriales como *Siemens Industry Support*, *Schneider Electric Download Center* y *ABB Library*. Sin embargo, para el desarrollo y demostración del proyecto, se generan documentos sintéticos ad hoc —partes de incidencia impresos y escritos a mano, formularios de orden de trabajo en distintos formatos y manual de operación de máquinas— para evaluar tanto la robustez del pipeline OCR ante variedad tipográfica y calidad de imagen, así como la capacidad de análisis documental del sistema.

El proyecto está asociado a la colaboración con Sopra Steria en el marco del Máster en Análisis y Visualización de Datos Masivos (UNIR). Sopra Steria sugirió algunas de las fuentes de datos

mencionadas, pero no proporcionó datos operativos reales de sus entornos de mantenimiento, por lo que solo se analizaron y se trabajó con los datos previamente mencionados, tanto para el desarrollo como para la validación de la solución.

7. Conclusiones

Como conclusión principal, se ha validado que la mayor aportación de este proyecto es la viabilidad técnica de una plataforma de inteligencia operativa totalmente desacoplada de servicios externos. Más allá del desarrollo de componentes individuales, el valor central reside en la integración exitosa de una arquitectura multiagente donde la orquestación (n8n) actúa como nexo de unión entre la percepción (extracción mediante OCR), el razonamiento (diagnóstico inteligente mediante RAG con Qwen2.5) y la observabilidad (Grafana). Se ha demostrado que es posible cerrar la brecha entre la documentación técnica estática y la operación diaria, proporcionando una herramienta de soporte a la decisión robusta, reproducible y basada íntegramente en software de código abierto. Respecto al objetivo general, se demuestra la viabilidad técnica del enfoque propuesto: la integración de extracción documental con OCR y modelos de lenguaje locales, validación determinista del dato, análisis mediante recuperación aumentada (RAG) y visualización operativa en Grafana conforma una cadena de valor del dato completa, reproducible y sin dependencias de servicios en la nube, que ninguna de las soluciones revisadas en el estado del arte combina de forma integrada. El grado de consecución de cada objetivo específico se detalla a continuación.

OE1. El análisis del dominio del mantenimiento industrial y de los conjuntos de datos de referencia AI4I 2020 y MetroPT-3 permitió definir el esquema JSON de incidencia, las variables monitorizadas (presiones, señales eléctricas, corriente del motor y temperatura del aceite) y los rangos de operación normal que sustentan tanto el módulo de validación como el manual técnico sintético empleado por el agente analista. Objetivo alcanzado.

OE2. La comparativa de soluciones OCR concluyó, con evidencia experimental sobre los formularios del Anexo A, en la selección justificada de PaddleOCR frente a Tesseract, al confirmarse que este último no alcanza la precisión necesaria para el reconocimiento de texto manuscrito en fotografías. Objetivo alcanzado.

OE3. Se implementó un módulo de ingesta y extracción documental funcional para las tres vías de entrada —formulario online, PDF e imagen con OCR—, capaz de producir el JSON estandarizado mediante un modelo de lenguaje local orquestado en n8n. Objetivo alcanzado.

OE4. Durante la implementación, el objetivo de normalización semántica evolucionó hacia un módulo de validación del JSON en dos capas —estructural (bloqueante) y semántica (alertas de coherencia con el manual)— acoplado a la recuperación documental por similitud (RAG). Si bien la unificación automática de nomenclaturas de activos quedó fuera del alcance final, la validación determinista cumple la función crítica de garantizar la calidad del dato antes del análisis, evitando la propagación de errores entre agentes señalada en el estado del arte. Objetivo cumplido con reorientación justificada.

OE5. El repositorio de incidencias se consolidó sobre MySQL 8.0, integrando en una única tabla los registros estructurados, los campos de validación (*validacion_ok*, *alertas_validacion*) y los errores de sistema, de forma consultable directamente por el *dashboard*. Objetivo alcanzado.

OE6 y OE7. Los objetivos de análisis temporal (OE6) y de índice de criticidad compuesto (OE7) se cubrieron en su dimensión descriptiva: el *dashboard* expone los indicadores de frecuencia, coste total estimado, tiempo medio de reparación (MTTR) y distribución por criticidad, y el sistema registra las tres magnitudes —frecuencia, duración y coste— que alimentarían un índice compuesto de priorización. El cálculo formal de dicho índice, así como el análisis de series temporales, patrones estacionales y correlaciones, quedan identificados como línea de trabajo futuro, al requerir un volumen histórico mayor que el disponible en la fase de validación. Objetivos cubiertos parcialmente.

OE8. El agente de diagnóstico técnico basado en RAG se implementó y validó: a partir del JSON de incidencia recupera los fragmentos relevantes del manual técnico y propone una acción correctiva con estimación de criticidad, tiempo y coste, ejecutándose íntegramente con un modelo de lenguaje local para preservar la privacidad de la información. Objetivo alcanzado.

OE9. El *dashboard* se construyó en Grafana con auto-provisionamiento (*dashboards-as-code*), integrando en una sola interfaz las métricas de negocio, el estado de validación, las alertas semánticas detectadas y el seguimiento de los errores de sistema. Objetivo alcanzado.

En síntesis, OpsInsight Analytics materializa una respuesta concreta al vacío identificado en el estado del arte. Los objetivos nucleares —ingesta multiformato, validación del dato, análisis documental con RAG, persistencia y visualización operativa— se alcanzaron y se validaron de extremo a extremo, mientras que las componentes analíticas más avanzadas —índice de criticidad compuesto y análisis temporal— quedan especificadas y parcialmente soportadas.

El resultado es un prototipo de producto mínimo viable, honesto en su alcance, que demuestra que es posible pasar de documentación operativa heterogénea a inteligencia analítica accionable mediante un enfoque local, reproducible y trazable, y que define una hoja de ruta clara hacia su evolución como producto profesional.

8. Limitaciones y prospectiva

8.1. Limitaciones

El desarrollo de OpsInsight Analytics en el contexto de este TFM enfrenta limitaciones de tres tipos principales.

En primer lugar, limitaciones de datos: la plataforma se evalúa sobre conjuntos de datos públicos (AI4I2020, MetroPT-3) y documentación sintética, no sobre datos reales de Sopra Steria. Esta decisión, motivada por la imposibilidad de acceder a datos operativos confidenciales durante la fase académica, limita la validación externa del sistema. Los resultados obtenidos pueden no generalizarse directamente a la variabilidad documental de un entorno real de producción, donde los formatos de partes de incidencia, las nomenclaturas de activos y los idiomas de los documentos son mucho más heterogéneos.

En segundo lugar, limitaciones técnicas: el uso de LLMs locales de 3B parámetros ejecutables en CPU, aunque suficiente para el alcance del proyecto, implica tiempos de inferencia considerablemente superiores a los de modelos en la nube. En entornos donde se procesen centenares de documentos por hora, este cuello de botella puede comprometer la viabilidad del pipeline. Asimismo, la arquitectura multiagente basada en n8n introduce complejidad de orquestación que, si bien facilita la modularidad, aumenta la superficie de error en los puntos de comunicación entre agentes y módulos.

En tercer lugar, limitaciones de alcance: el prototipo desarrollado es un demostrador funcional, no un sistema listo para producción. No incorpora mecanismos de autenticación, control de acceso por roles, gestión de versiones del modelo ni integración con sistemas CMMS o ERP existentes. Estas capacidades serían necesarias para su adopción en un entorno empresarial real.

8.2.Trabajo futuro

Las limitaciones identificadas apuntan directamente a las líneas de trabajo futuro más relevantes.

La validación con datos reales de Sopra Steria constituye el paso inmediato más importante. La incorporación de partes de incidencia reales, con toda su variabilidad de formato y contenido, permitiría medir con precisión la robustez del pipeline de extracción y normalización, e identificar los tipos de documento que requieren tratamiento específico.

De todas maneras, tal y como se explica en la sección 5.2.2, con los datos sintéticos generados por nosotros mismos (presentados en el Anexo A), ya se ha detectado que el OCR utilizado en la ingesta es mejorable, especialmente para fotografías de informes manuscritos. A continuación, se enumeran vías de trabajo que podrían mejorar los resultados de la ingesta de imágenes:

- Preprocesado de las imágenes, incluyendo su transformación a escala de grises o incluso a imagen binaria que aumente el contraste. También se podrían detectar los bordes de la hoja y rectificar la imagen para que el texto no aparezca inclinado.
- Experimentar con otros algoritmos de OCR. Dada la estructura modular implementada, es muy sencillo configurar la API para que, al recibir las peticiones de n8n, utilice otro OCR. Como se comentó en el estado del arte, potenciales alternativas son Surya, Docling, Easy, Mistral, Qwen2.5-VL, InternVL y GOT-OCR 2.0.
- Añadir un bloque de corrección del JSON generado. Hay algunos errores comunes que podrían verificarse y corregirse automáticamente. Por ejemplo:
 - “l”, “O”, “Z” o “T” en campos numéricos pueden ser “0”, “1”, “2” o “7”.
 - Los nombres de los operarios y de las máquinas podrían cotejarse con una base de datos con los nombres válidos.
 - Letras acentuadas con “^” o “”” o precedidas o seguidas de comillas simples o dobles son muy probablemente letras acentuadas mal interpretadas.
 - Podría validarse la existencia de todas las palabras con un diccionario. Esto permitiría detectar palabras en las que haya cambiado una “u” por una “v” o una “m” por una “n” y cambiarla por la palabra existente más similar de entre las palabras del diccionario.

A más largo plazo, la extensión del sistema hacia un escenario de monitorización en tiempo real —integrando *streams* de datos de sensores IoT junto con la documentación operativa— abriría la puerta a un sistema de mantenimiento predictivo unificado que combina análisis documental e histórico con detección temprana de anomalías. Esta convergencia representa la dirección más prometedora para la evolución de OpsInsight Analytics como producto profesional.

En el caso de evolucionar hacia una solución real y operativa, sería crítico revisar la arquitectura de la solución para garantizar su seguridad y escalabilidad. Algunas ideas de trabajo con respecto a estos puntos serían:

- Implementar medidas de seguridad que filtren el tráfico entrante y saliente.
- Realizar tests de *query injection* que podrían incluirse en los reportes de forma malintencionada para que los agentes realicen acciones que no deberían.
- Hay que asegurar que la base de datos está anonimizada y se dispone de un sistema de accesos basado en roles que aumente la seguridad.
- Implementar la solución en un servidor suficientemente potente y debidamente configurado que permita ejecutar los LLM en local utilizando GPUs para aumentar la velocidad de respuesta. También pueden usarse APIs de pago y usar LLMs en la nube.

Referencias bibliográficas

- ABBYY. (s. f.). *FineReader PDF* [Software]. Recuperado 19 de mayo de 2026, de <https://pdf.abbyy.com/>
- Activepieces. (2022). *Activepieces: Open source AI automation* [Software]. <https://www.activepieces.com/>
- American Bankers Association. (s. f.). *1950-1974: A history of America's banks and the ABA*. Recuperado 19 de mayo de 2026, de <https://www.aba.com/about-us/our-story/aba-history/1950-1974>
- Baek, Y., Lee, B., Han, D., Yun, S., & Lee, H. (2019). *Character region awareness for text detection* (arXiv:1904.01941). arXiv. <https://doi.org/10.48550/arXiv.1904.01941>
- Baidu. (s. f.). *PaddleOCR* [Software]. Recuperado 19 de mayo de 2026, de <https://aistudio.baidu.com/paddleocr?lang=en>
- Biró, A., Szilágyi, S. M., & Szilágyi, L. (2023). Optimal training dataset preparation for AI-supported multilanguage real-time OCRs using visual methods. *Applied Sciences*, 13(24), 13107. <https://doi.org/10.3390/app132413107>
- Bispo, N. (2025, septiembre 10). Best open source OCR tools & models for developers in 2026. *Unstract Blog*. <https://unstract.com/blog/best-opensource-ocr-tools/>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. <https://arxiv.org/abs/2005.14165>
- Cheng, X., Chaw, J. K., Goh, K. M., Ting, T. T., Sahrani, S., Ahmad, M. N., Abdul Kadir, R., & Ang, M. C. (2022). Systematic literature review on visual analytics of predictive maintenance in the manufacturing industry. *Sensors*, 22(17), 6321. <https://doi.org/10.3390/s22176321>

- Criado, J. S. (2026, febrero 20). Mejores modelos open source de OCR: Ranking completo [2026]. *Javadex*. <https://www.javadex.es/blog/mejores-modelos-open-source-ocr-reconocimiento-texto-2026>
- Dmitrievna, Y., & Parsadanyan, E. (2025, diciembre 22). Multi-agent systems: Frameworks & step-by-step tutorial. *n8n Blog*. <https://blog.n8n.io/multi-agent-systems/>
- Du, Y., Li, C., Guo, R., Yin, X., Liu, W., Zhou, J., Bai, Y., Yu, Z., Yang, Y., Dang, Q., & Wang, H. (2020). *PP-OCR: A practical ultra lightweight OCR system* (arXiv:2009.09941). arXiv. <https://doi.org/10.48550/arXiv.2009.09941>
- DuckDB Foundation. (2024). *DuckDB documentation*. <https://duckdb.org/docs/>
- Galar, D., Thaduri, A., Catelani, M., & Ciani, L. (2015). Context awareness for maintenance decision making: A diagnosis and prognosis approach. *Measurement*, 67, 137–150. <https://doi.org/10.1016/j.measurement.2015.01.015>
- Goldberg, E. (1931). *Statistical machine* (United States Patent No. US1838389A). <https://patents.google.com/patent/US1838389A/en>
- Google Books. (s. f.). Recuperado 19 de mayo de 2026, de <https://books.google.com/>
- Google Trends. (s. f.). Recuperado 19 de mayo de 2026, de <https://trends.google.com/trends/>
- Haddad, G., Sandborn, P. A., & Pecht, M. G. (2012). An options approach for decision support of systems with prognostic capabilities. *IEEE Transactions on Reliability*, 61(4), 872–883. <https://doi.org/10.1109/TR.2012.2220699>
- Hector, I., & Panjanathan, R. (2024). Predictive maintenance in Industry 4.0: A survey of planning models and machine learning techniques. *PeerJ Computer Science*, 10, e2016. <https://doi.org/10.7717/peerj-cs.2016>
- History of Information. (s. f.). Recuperado 19 de mayo de 2026, de <https://www.historyofinformation.com/detail.php?entryid=885>
- Keren, G., & Schuller, B. (2017). *Convolutional RNN: An enhanced model for extracting features from sequential data* (arXiv:1602.05875). arXiv. <https://doi.org/10.48550/arXiv.1602.05875>

- Kim, G., Hong, T., Yim, M., Nam, J., Park, J., Yim, J., Hwang, W., Yun, S., Han, D., & Park, S. (2022). *OCR-free document understanding transformer* (arXiv:2111.15664). arXiv. <https://doi.org/10.48550/arXiv.2111.15664>
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25. https://papers.nips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html
- Lecun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324. <https://doi.org/10.1109/5.726791>
- Li, M., Lv, T., Chen, J., Cui, L., Lu, Y., Florencio, D., Zhang, C., Li, Z., & Wei, F. (2022). *TrOCR: Transformer-based optical character recognition with pre-trained models* (arXiv:2109.10282). arXiv. <https://doi.org/10.48550/arXiv.2109.10282>
- Lin, J., Ren, X., Zhang, Y., Liu, G., Wang, P., Yang, A., & Zhou, C. (2022). *Transferring general multimodal pretrained models to text recognition* (arXiv:2212.09297). arXiv. <https://doi.org/10.48550/arXiv.2212.09297>
- Matzka, S. (2020). *AI4I 2020 Predictive Maintenance Dataset* [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5HS5C>
- Mobley, R. K. (2002). *An introduction to predictive maintenance* (2.^a ed.). Butterworth-Heinemann.
- Modal. (s. f.). *8 top open-source OCR models compared: A complete guide*. Recuperado 19 de mayo de 2026, de <https://modal.com/blog/8-top-open-source-ocr-models-compared>
- n8n GmbH. (2019). *n8n: Flexible AI workflow automation* [Software]. <https://n8n.io/>
- NASA. (2007). *PCoE prognostics data repository* [Dataset]. NASA Ames Research Center. <https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/>
- Ollama. (2023). *Ollama* [Software]. <https://ollama.com/>

- Pech, M., Vrchota, J., & Bednář, J. (2021). Predictive maintenance and intelligent sensors in smart factory: Review. *Sensors*, 21(4), 1470. <https://doi.org/10.3390/s21041470>
- Raasveldt, M., & Mühleisen, H. (2019). DuckDB: An embeddable analytical database. *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)*, 1981–1984. <https://doi.org/10.1145/3299869.3320212>
- Raza, A., Munir, K., Almutairi, M., Younas, M., & Farhan, M. S. (2023). Predicting network anomalies and equipment malfunctions: A literature review of machine learning approaches. *IEEE Access*, 11, 18149–18174. <https://doi.org/10.1109/ACCESS.2023.3245668>
- Smith, R. (2007). An overview of the Tesseract OCR engine. *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, 2, 629–633. <https://doi.org/10.1109/ICDAR.2007.4376991>
- Sopra Steria. (2025). *Herramientas de referencia para el entorno de automatización* [Documento interno no publicado].
- Stamatis, D. H. (2003). *Failure mode and effect analysis: FMEA from theory to execution* (2.^a ed.). ASQ Quality Press.
- Taoufyq, H., Guemmat, K. E., Mansouri, K., & Akef, F. (2025). Predictive maintenance approaches: A systematic literature review. *Journal of Industrial Engineering and Management*, 18(3), 427–458. <https://doi.org/10.3926/jiem.8537>
- Tauschek, G. (1935). *Reading machine* (United States Patent No. US2026330A). <https://patents.google.com/patent/US2026330A/en>
- tesseract-ocr. (2026). *Tesseract-ocr/tesseract* [C++]. <https://github.com/tesseract-ocr/tesseract> (Obra original publicada en 2014)
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., & Lample, G. (2023). *LLaMA: Open and efficient foundation language models* (arXiv:2302.13971). arXiv. <https://arxiv.org/abs/2302.13971>
- US National Archives. (2015, diciembre 15). *Reading and sorting mail automatically* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=V4LJs2ZoDR4>

- Veloso, B., Gama, J., Ribeiro, B., & Pereira, P. (2022). MetroPT: A benchmark dataset for predictive maintenance. *Scientific Data*, 9, 764. <https://doi.org/10.1038/s41597-022-01877-3>
- Wikiel, K. (s. f.). *OCR SOTA Router 2026: Choose OCR by output contract*. CodeSOTA. Recuperado 19 de mayo de 2026, de <https://www.codesota.com/ocr>
- Zhou, X., Yao, C., Wen, H., Wang, Y., Zhou, S., He, W., & Liang, J. (2017). *EAST: An efficient and accurate scene text detector* (arXiv:1704.03155). arXiv. <https://doi.org/10.48550/arXiv.1704.03155>
- Zonta, T., da Costa, C. A., da Rosa Righi, R., de Lima, M. J., da Trindade, E. S., & Li, G. P. (2020). Predictive maintenance in the Industry 4.0: A systematic literature review. *Computers & Industrial Engineering*, 150, 106889. <https://doi.org/10.1016/j.cie.2020.106889>

9. ÍNDICE DE ACRÓNIMOS Y ABREVIATURAS

ACRÓNIMO / ABREVIATURA	DESCRIPCIÓN
ARIMA	AUTOREGRESSIVE INTEGRATED MOVING AVERAGE — MODELO ESTADÍSTICO DE SERIES TEMPORALES PARA PREDICCIÓN
CMMS	COMPUTERIZED MAINTENANCE MANAGEMENT SYSTEM — SISTEMA INFORMATIZADO DE GESTIÓN DEL MANTENIMIENTO
CPU	CENTRAL PROCESSING UNIT — UNIDAD CENTRAL DE PROCESAMIENTO
CRISP-DM	CROSS-INDUSTRY STANDARD PROCESS FOR DATA MINING — PROCESO ESTÁNDAR ENTRE SECTORES PARA MINERÍA DE DATOS
CSV	COMMA-SEPARATED VALUES — VALORES SEPARADOS POR COMAS (FORMATO DE ARCHIVO DE TEXTO PLANO)

DBSCAN	DENSITY-BASED SPATIAL CLUSTERING OF APPLICATIONS WITH NOISE — ALGORITMO DE AGRUPAMIENTO BASADO EN DENSIDAD
MYSQL	SISTEMA DE GESTIÓN DE BASES DE DATOS RELACIONAL (RDBMS) DE ALTO RENDIMIENTO, ORIENTADO A TABLAS Y OPTIMIZADO PARA PROCESAMIENTO TRANSACCIONAL (OLTP) Y CONSULTAS SQL
EDA	EXPLORATORY DATA ANALYSIS — ANÁLISIS EXPLORATORIO DE DATOS
ERP	ENTERPRISE RESOURCE PLANNING — PLANIFICACIÓN DE RECURSOS EMPRESARIALES
GPU	GRAPHICS PROCESSING UNIT — UNIDAD DE PROCESAMIENTO GRÁFICO
IOT	INTERNET OF THINGS — INTERNET DE LAS COSAS
JSON	JAVASCRIPT OBJECT NOTATION — FORMATO LIGERO DE INTERCAMBIO DE DATOS BASADO EN TEXTO
KPI	KEY PERFORMANCE INDICATOR — INDICADOR CLAVE DE RENDIMIENTO
LLM	LARGE LANGUAGE MODEL — MODELO DE LENGUAJE DE GRAN ESCALA
LSTM	LONG SHORT-TERM MEMORY — ARQUITECTURA DE RED NEURONAL RECURRENTE PARA SECUENCIAS TEMPORALES
MAS	MULTI-AGENT SYSTEM — SISTEMA MULTIAGENTE
MCP	MODEL CONTEXT PROTOCOL — PROTOCOLO ESTÁNDAR DE INTEROPERABILIDAD ENTRE AGENTES LLM Y HERRAMIENTAS EXTERNAS
MTBF	MEAN TIME BETWEEN FAILURES — TIEMPO MEDIO ENTRE FALLOS
MTTR	MEAN TIME TO REPAIR — TIEMPO MEDIO DE REPARACIÓN

NER	NAMED ENTITY RECOGNITION — RECONOCIMIENTO DE ENTIDADES NOMBRADAS
OCR	OPTICAL CHARACTER RECOGNITION — RECONOCIMIENTO ÓPTICO DE CARACTERES
PDF	PORTABLE DOCUMENT FORMAT — FORMATO DE DOCUMENTO PORTÁTIL
RAG	RETRIEVAL-AUGMENTED GENERATION — GENERACIÓN AUMENTADA POR RECUPERACIÓN DE INFORMACIÓN
RAM	RANDOM ACCESS MEMORY — MEMORIA DE ACCESO ALEATORIO
RCNN	RECURRENT CONVOLUTIONAL NEURAL NETWORK — RED NEURONAL CONVOLUCIONAL RECURRENTE
RGPD	REGLAMENTO GENERAL DE PROTECCIÓN DE DATOS (UE 2016/679)
TFM	TRABAJO DE FIN DE MÁSTER
UCI	UNIVERSITY OF CALIFORNIA, IRVINE (REPOSITORIO DE DATASETS DE REFERENCIA PARA ML)